



L3 informatique

Course Notes
Software Engineering

Course 3
Modeling
Object and Class Diagram

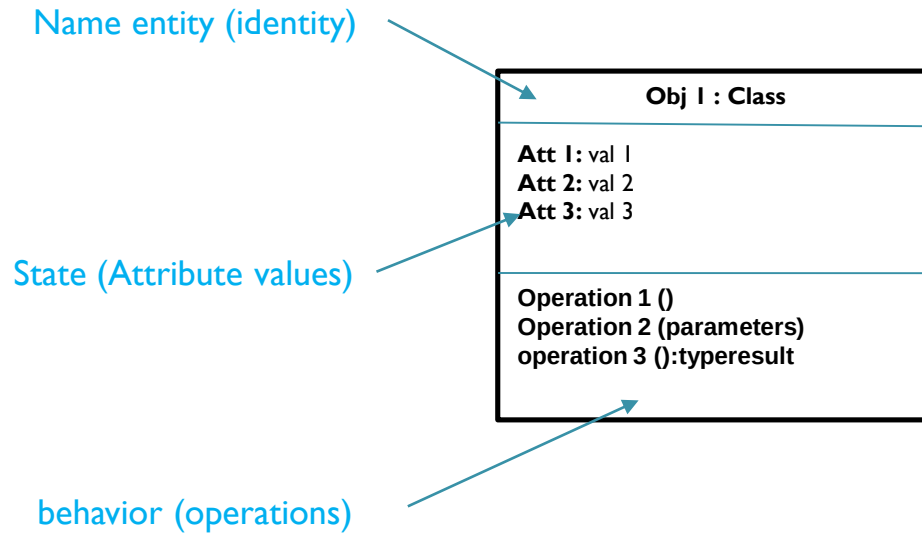
Presented by
Mr KHELIFA N.

Class Diagram

- **The class diagram is considered the most important in object-oriented modeling; it is the only mandatory one in such modeling.**
- The class diagram shows the internal structure of a system.
- It provides an abstract representation of the system's objects that will interact to realize the use cases.
- It is a static view, since the time factor in the system's behavior is not taken into account.
- The class diagram allows the modeling of the system's classes and their relationships, independently of any specific programming language.

Object

- An entity with a well-defined boundary and identity, which encapsulates both state and behavior.



Example

e3: Student
Id: 11129 LastName: Salim FirstName: Amir Program: Bachelor's Degree YearOfStudy: 3
CalculateAverage() ReviewCourses()

Name entity
(identity)

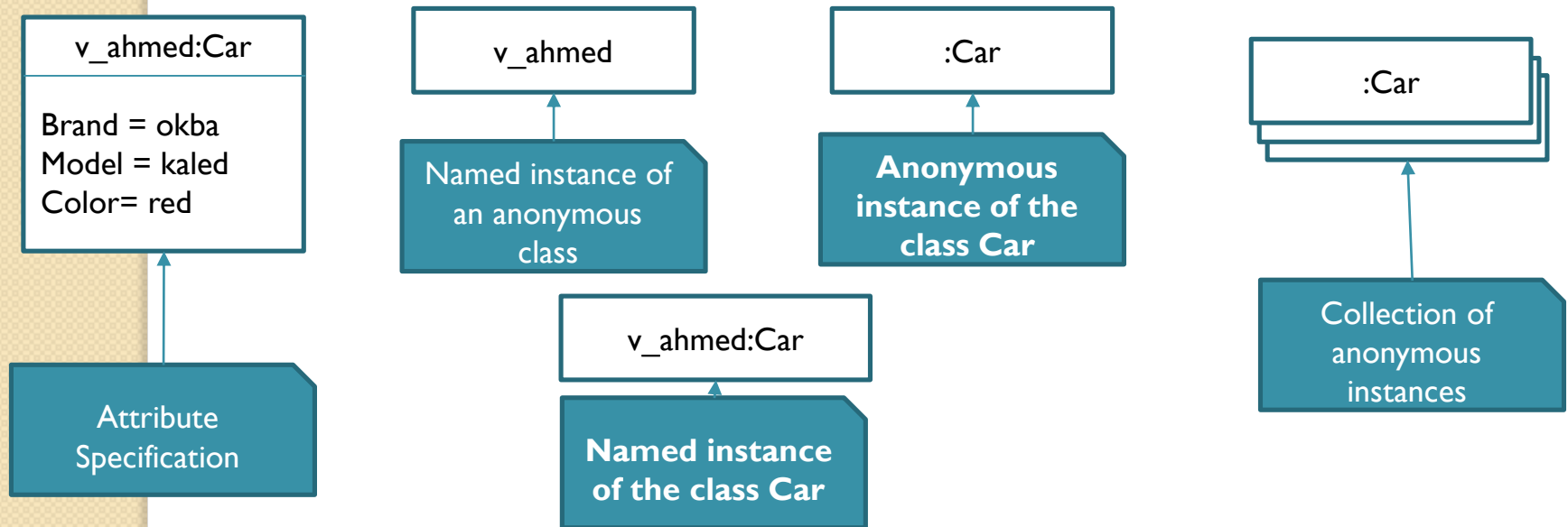
State (Attribute
values)

behavior
(operations)

MyAccountA : Account
Id: 11129 currency: euro balance: 1000
deposit(amount:float) withdraw(amount:float) balance():float

JoeSmith:Person
name="Joe Smith" birthdate=21 October 1983

Example of How to Model Objects



Exemple object I

e1: Student
Id: 11121 LastName: mohamed FirstName: Amir Program: Bachelor's Degree YearOfStudy: 3
CalculateAverage() ReviewCourses()

e2: Student
Id: 11125 LastName: okba FirstName: mire Program: Bachelor's Degree YearOfStudy: 3
CalculateAverage() ReviewCourses()

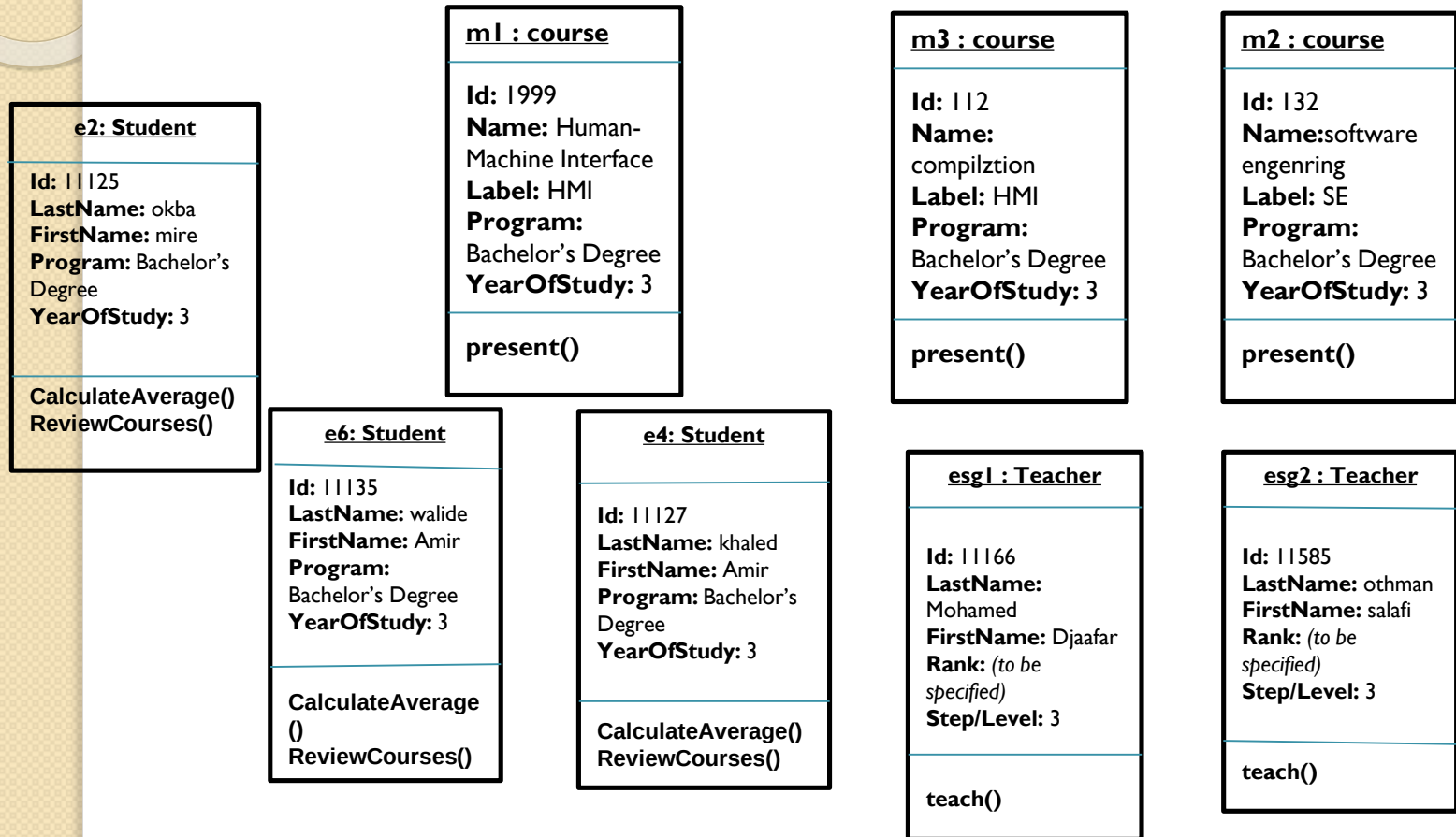
e3: Student
Id: 11129 LastName: Salim FirstName: Amir Program: Bachelor's Degree YearOfStudy: 3
CalculateAverage() ReviewCourses()

e4: Student
Id: 11127 LastName: khaled FirstName: Amir Program: Bachelor's Degree YearOfStudy: 3
CalculateAverage() ReviewCourses()

e5: Student
Id: 11130 LastName: nafaa FirstName: Amir Program: Bachelor's Degree YearOfStudy: 3
CalculateAverage() ReviewCourses()

e6: Student
Id: 11135 LastName: walide FirstName: Amir Program: Bachelor's Degree YearOfStudy: 3
CalculateAverage() ReviewCourses()

Exemple of object 2



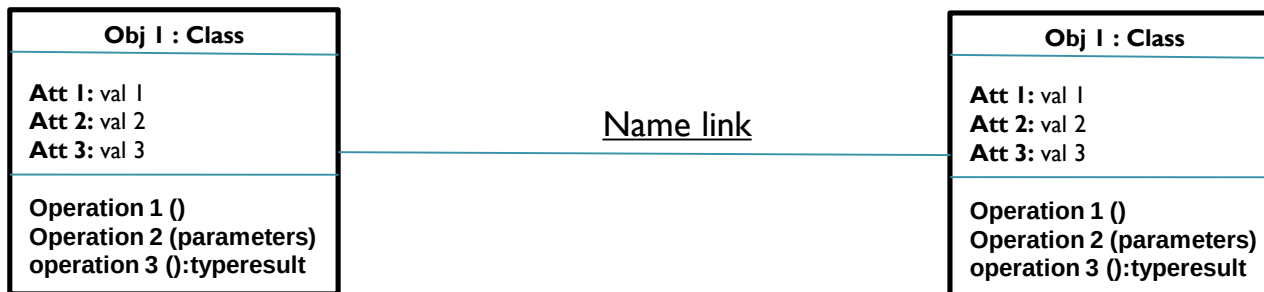
Links

- **Objects are connected by instances of associations, called links.**

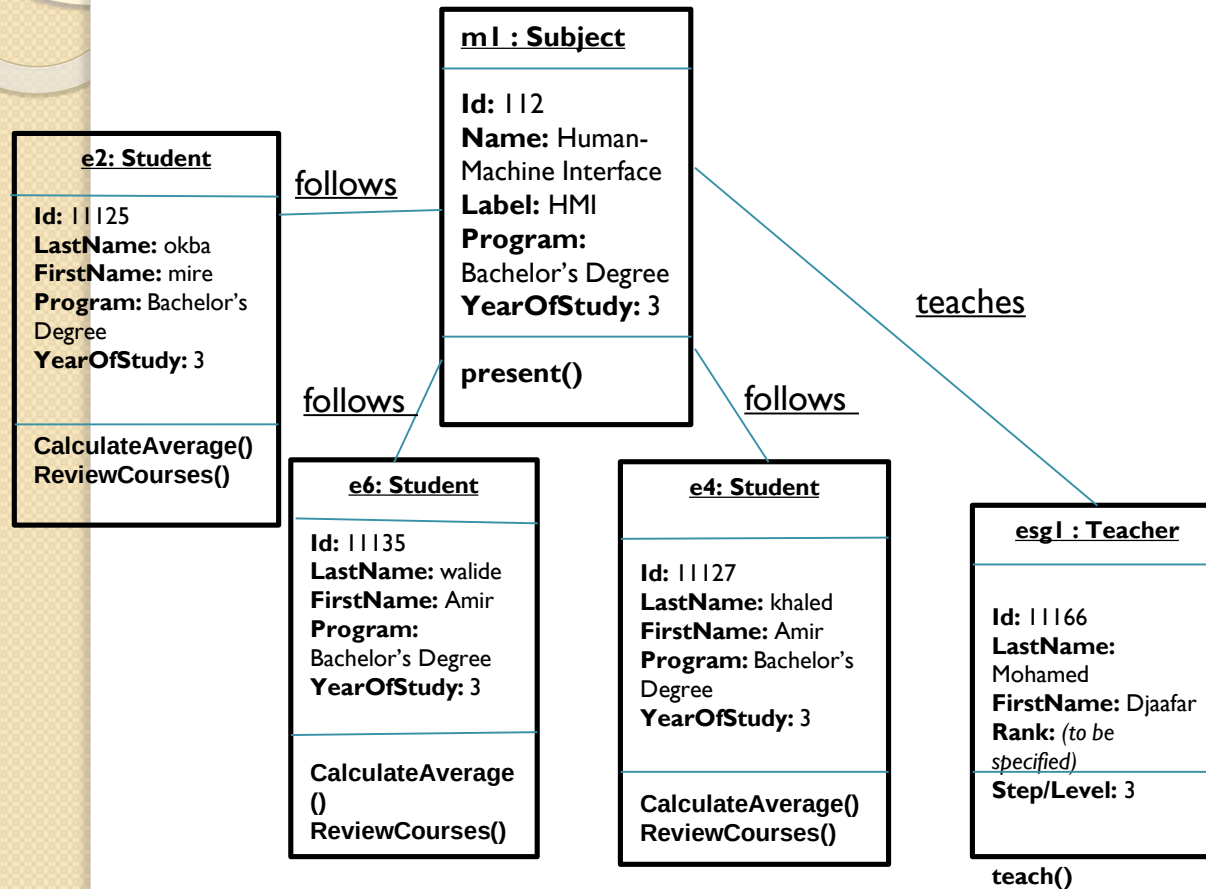
A link represents a relationship between objects at a given moment in time.

If a link has a name, it must be underlined.

- **Note:** the multiplicity at the ends of links is always 1.



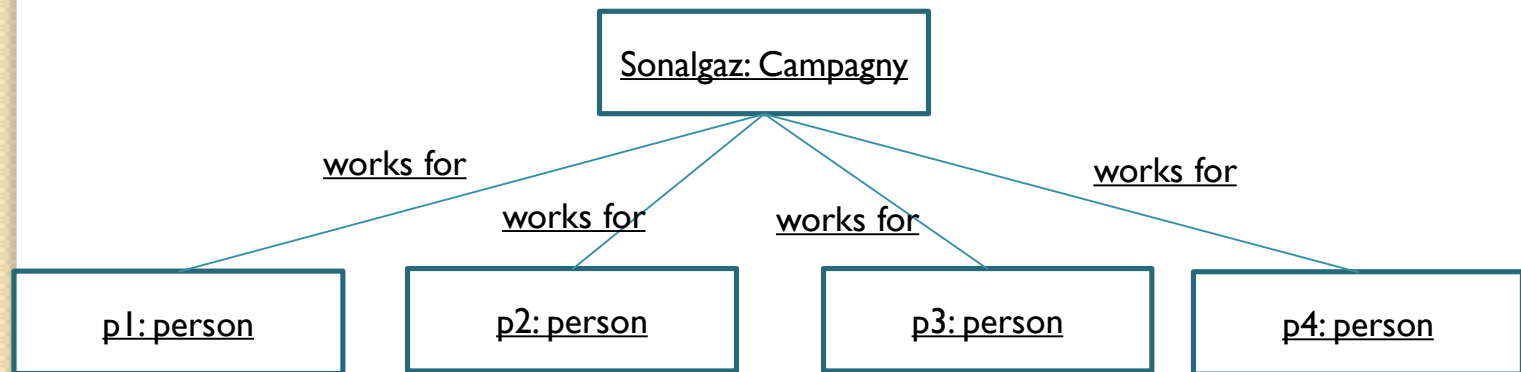
Example 1: Object Diagram



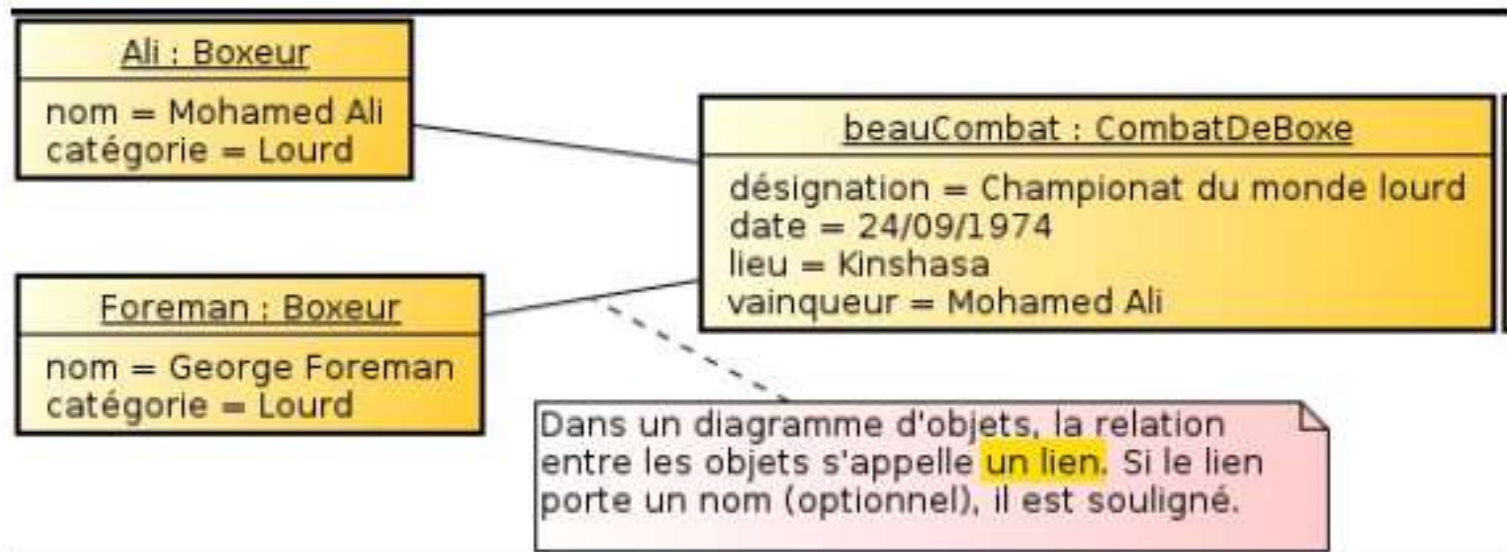
object diagram

- An **object diagram** represents objects (i.e., instances of classes) and their links (i.e., instances of relationships) to provide a static view of the state of a system at a given moment.
- An object diagram can be used to:
 - illustrate the class model by showing an example that explains the model;
 - specify certain aspects of the system by highlighting details that may not be visible in the class diagram;
 - express an exception by modeling specific cases or non-generalizable knowledge that are not represented in a class diagram;
 - take a **snapshot** of a system at a particular moment.
 - An object diagram does not show interactions between objects.
- The **class diagram** models the rules, while the **object diagram** models the facts.

Example : Object Diagram

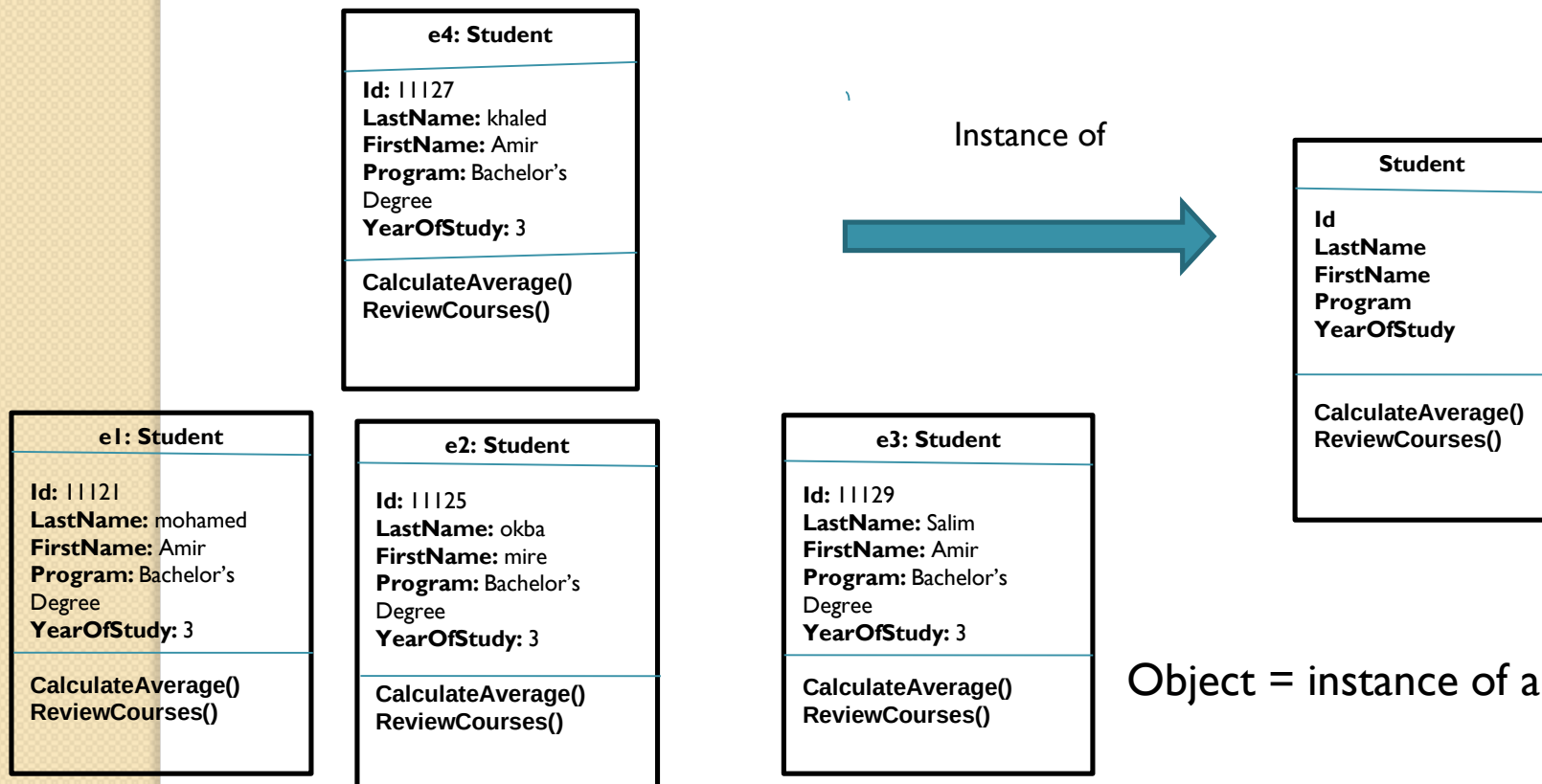


Exemple 2 : Diagramme objet



class

- A class is a description of a set of objects that share the same characteristics (attributes, operations, etc.) and the same semantics



A class

- **A class is an abstract concept representing various elements such as:**
 - **Concrete elements** (e.g., airplanes),
 - **Abstract elements** (e.g., purchase orders for goods or services),
 - **Application components** (e.g., dialog box buttons),
 - **Data structures** (e.g., hash tables),
 - **Behavioral elements** (e.g., tasks), etc.

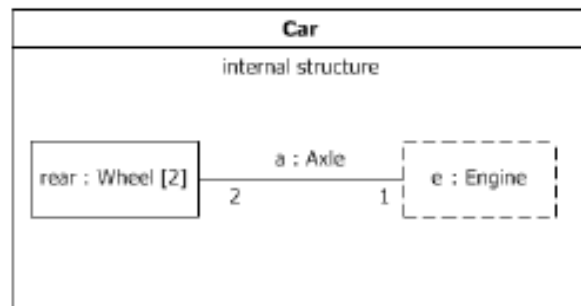
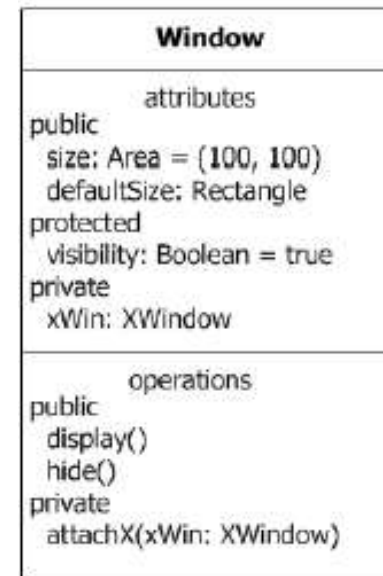
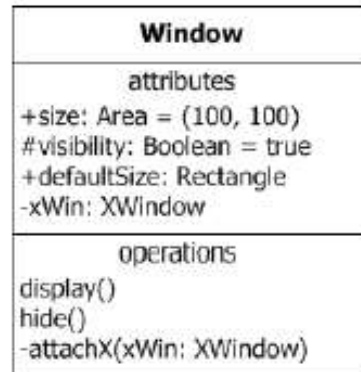
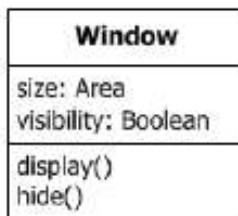
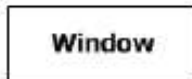
Class Structure

- **A class has four compartments:**
 - **Name** (mandatory)
 - **Attributes**
 - **Operations**
 - **Receptions**
 - **Internal structure**
- A class may also include optional compartments, as described for classifiers in general. The compartments are represented as horizontally juxtaposed rectangles.

Class name
attributes
Operations
Responsibility and/or Exception

Class name
-Attribute_1: type 1 -Attribute_2: type 2
+operations_1():type1 +operations_2(): void

Example



Class Name

- The class name must evoke the concept described by the class.
It begins with an uppercase letter.
Additional information such as the name of the model's author, the date, etc., may be added.
To indicate that a class is abstract, the keyword *abstract* must be added.
- The basic syntax for declaring a class name is as follows:

[<Nom_du_paquetage_1>::...::<Nom_du_paquetage_N>]

<Name of class>

[{ [abstract], [<auteur>], [<date>], ... }]

Examples

- Class Wheel

Wheel

- Class Wheel contained in package Vehicle

Vehicle :: Wheel

- The Wheel class with information

Wheel {abstract, author::ahmed }

Attributes (Property)

- **A named property of a classifier that describes a range of values that the instances of the classifier can take.**
- The name of an attribute begins with a lowercase letter.
- **Example:**
Name, firstName, address, email, number, etc., are the named properties of the class *Person*.

Example Attributes

Name	Attribute
Dog	Name, breed, age, color
Account	Number, type, balance, credit line
Phone	Number, brand, ringtone, directory, operator
Car	License plate, brand, color, speed

Attributes

- The general form:
- $\langle \textit{property} \rangle ::=$
- $[\langle \textit{visibility} \rangle] [\textit{'/'}] \textcolor{red}{\langle \textit{name} \rangle} [\textit{'.'} \langle \textit{prop-type} \rangle]$
 $[\textit{'['} \langle \textit{multiplicity-range} \rangle \textit{']'}] [\textit{'='} \langle \textit{default} \rangle]$
 $[\textit{'{'} \langle \textit{prop-modifier} \rangle [\textit{' , ' } \langle \textit{prop-modifier} \rangle]^* \textit{'}} \textit{'}]$

[<visibility>] ['/'] <name> [':' <prop-type>] ['[' <multiplicity-range> ']'] ['=' <default>] ['{' <prop-modifier > [',' <prop-modifier >]* '}']

- **<visibility>**: denotes the visibility of the property.
<visibility> ::= '+' | '-' | '#' | '~'
- '/' indicates that the property is derived.
<name> is the name of the property.
<prop-type> is the name of the classifier that represents the type of the property.
<multiplicity-range> specifies the multiplicity range of the property. If this term is omitted, it implies a multiplicity of 1 (exactly one).
<default> is an expression that corresponds to the default value(s) of the property.
<prop-modifier> designates a modifier that applies to the property.
- **<prop-modifier>** ::= 'readOnly' | 'union' | 'subsets' <property-name> | 'redefines' <property-name> | 'ordered' | 'unordered' | 'unique' | 'nonunique' | 'seq' | 'sequence' | 'id' | <prop-constraint>

<prop-modifier>

- **readOnly** means that the property is read-only.
 - **union** means that the property is a union derived from its subsets.
 - **subsets** **<property-name>** means that the property is a proper subset of the property identified by **<property-name>**, where **<property-name>** may be qualified.
 - **redefines** **<property-name>** means that the property redefines an inherited property identified by **<property-name>**, where **<property-name>** may be qualified.
 - **ordered** means that the property is ordered.
 - **unordered** means that the property is unordered.
 - **unique** means that there are no duplicates in a multi-valued property.
 - **nonunique** means that duplicates may exist in a multi-valued property.
 - **seq** or **sequence** means that the property represents an ordered bag.
 - **id** means that the property is part of the identifier of the class.
- <prop-constraint>** is an expression that specifies a constraint applicable to the property.

Derived attributes

- **Derived attributes** can be computed from other attributes and calculation formulas.
- During design, a derived attribute may be used as a placeholder until you can determine the rules to apply to it.
- Derived attributes are symbolized by adding a “/” in front of their name

Primitive type

- **Integer:** An instance of *Integer* is a value within the (infinite) set of integers.
- **Boolean:** An instance of *Boolean* is one of the predefined values *true* or *false*.
- **String:** An instance of *String* represents a sequence of characters.
- **UnlimitedNatural:** The unbounded value is denoted using an asterisk (“*”).
- **Real (float):** An instance of *Real* is a value within the (infinite) set of real numbers.

Example

- name: String
- shape: Rectangle
- +size: Integer[2..5]
- /area: Integer[5] {readOnly}
- -height: Integer = 5
- width: Integer

Operation

- Service that can be requested from any object of the class
- Behavior common to all objects of the class
- Method = implementation of the operation

- **Examples of operations**

Name	Attribute	Operation
Dog	Name, breed, age, color	Bark, fetch the stick, bite, play dead
Account	Number, type, balance, credit line	Withdrawal, transfer, deposit, balance inquiry
Phone	Number, brand, ringtone, directory, operator	Call, answer a call, send SMS, charge
Car	License plate, brand, color, speed	Turn, accelerate, stop, refuel, honk

Operation

- *general syntaxes*
- `[<visibility>] <name> ‘(’ [<parameter-list>] ‘)’ [‘.’ [<return-type>] [‘[’
<multiplicity-range> ‘]’] [‘{’ <oper-property> [‘,’ <oper-property>]*
}’]`

`[<visibility>] ['/'] <name> [':' <prop-type>] ['[' <multiplicity-range> ']' ] ['=' <default>] ['{' <prop-modifier > [',' <prop-modifier >]* '}' ]`

<visibility> is the visibility of the operation.

<visibility> ::= '+' | '-' | '#' | '~'

<name> is the name of the operation.

<parameter-list> is a list of operation parameters in the following format:

<parameter-list> ::= <parameter> [',' <parameter>] where <parameter>*

<return-type> is the type of the return result parameter, if the operation defines one.

<multiplicity-range> is the multiplicity of the return type.

<oper-property> indicates the properties of the operation.

<oper-property> ::= 'redefines' <oper-name> | 'query' | 'ordered' | 'unordered' | 'unique' | 'non-unique' | 'bag' | 'sequence' | <oper-constraint>

Oper_property

- **redefines <oper-name>** means that the operation redefines an inherited operation identified by <oper-name>, where <oper-name> may be qualified.
- **query** means that the operation does not modify the state of the system.
- **ordered** applies when there is a multi-valued return parameter and means that its values are ordered.
- **unordered** applies when there is a multi-valued return parameter and means that its values are not ordered.
- **unique** applies when there is a multi-valued return parameter and means that its values contain no duplicates.
- **nonunique** applies when there is a multi-valued return parameter and means that its values may contain duplicates.
- **seq** or **sequence** applies when there is a multi-valued return parameter and means that its values form an ordered bag.
- **<oper-constraint>** is a constraint that applies to the operation.
The parameter list may be omitted.

parameter

<parameter> ::= [<direction>] <parameter-name> ':' <type-expression> ['[' <multiplicity-range> ']'] ['=' <default>] ['{' <parm-property> [',' <parm-property>]* '}']
Or:

<direction> ::= 'in' | 'out' | 'inout' (defaults to 'in' if omitted).

<parameter-name> is the name of the parameter.

<type-expression> is an expression that specifies the type of the parameter.

<multiplicity-range> is the multiplicity of the parameter.

<default> is an expression that specifies the default value of the parameter.

<parm-property> indicates additional property values that apply to the parameter.

Property of a parameter

<parm-property> ::= 'ordered' | 'unordered' | 'unique'

- **ordered** applies when there is a multi-valued parameter and means that its values are ordered.
- **unordered** applies when there is a multi-valued parameter and means that its values are not ordered.
- **unique** applies when there is a multi-valued parameter and means that its values contain no duplicates.

exemples

- display ()
- -hide ()
- +createWindow (location: Coordinates, container: Container [0..1]):Window
- +toString (): String
- +calculer(in a: integer, in b: boolean [5], out c: Real) : Real



Relationship between classes

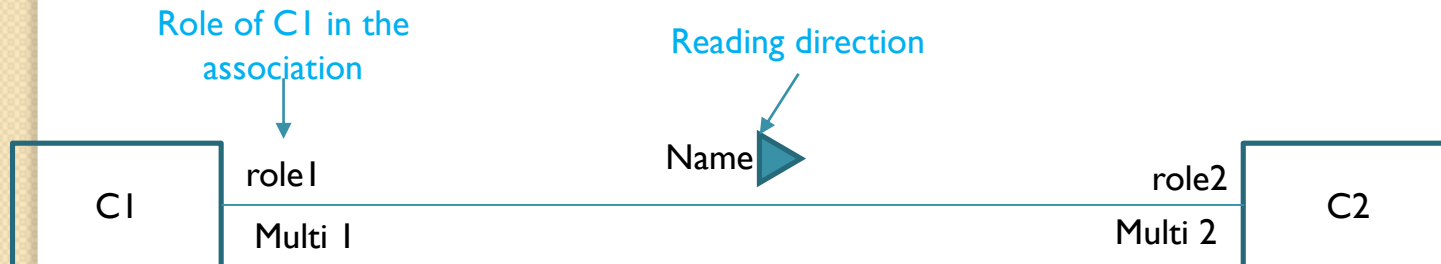
- **Association (binary and n-ary)**
 - Aggregation
 - Composition
- **Generalization**
- **Dependency**

Association

- **Semantic relationship between two or more classifiers, which implies links between their instances.**

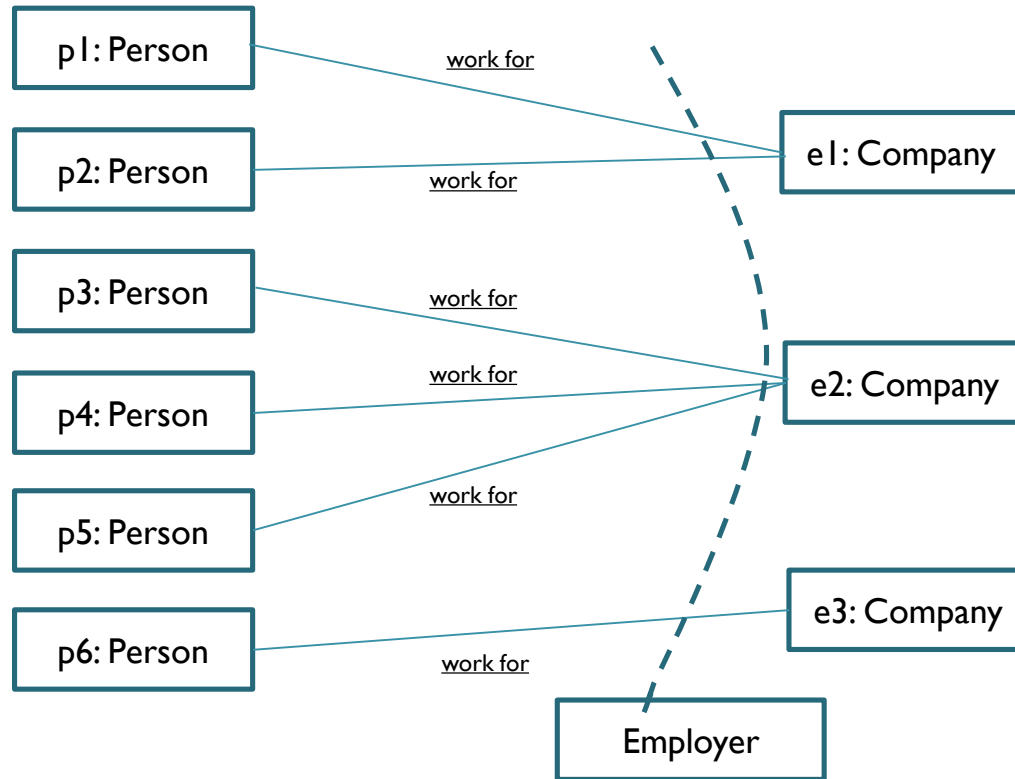
It represents an association through a solid line.

- Each association has:
 - **Name:** a verbal form.
 - **Roles:** a nominal form describing the status of the class in the association. It may include an access mode.
 - **Multiplicity:** a number (or a range of numbers) of instance(s) that the association may involve.

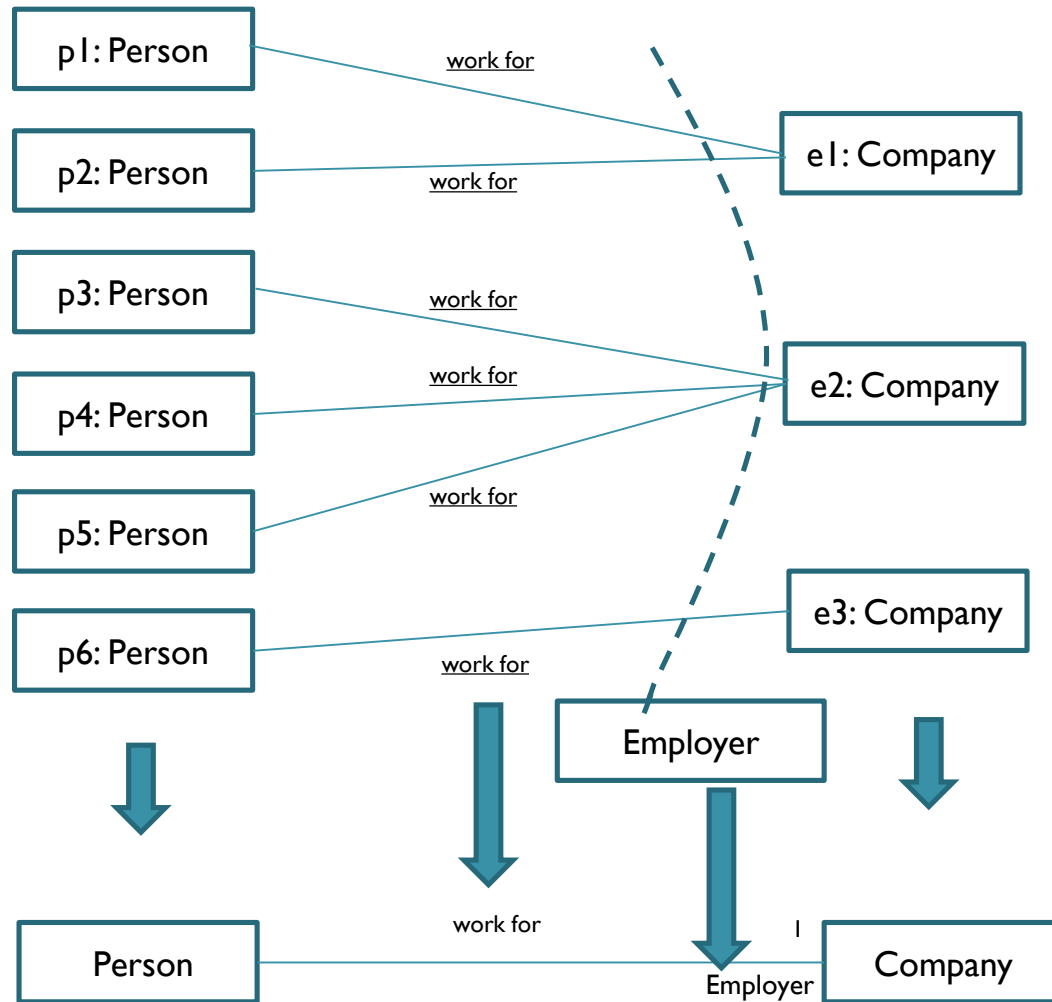


Number of instances of C1 that can be linked to an instance of C2 (exemple : (1 ; 0..1 ; 1..* ; *))

Example 1

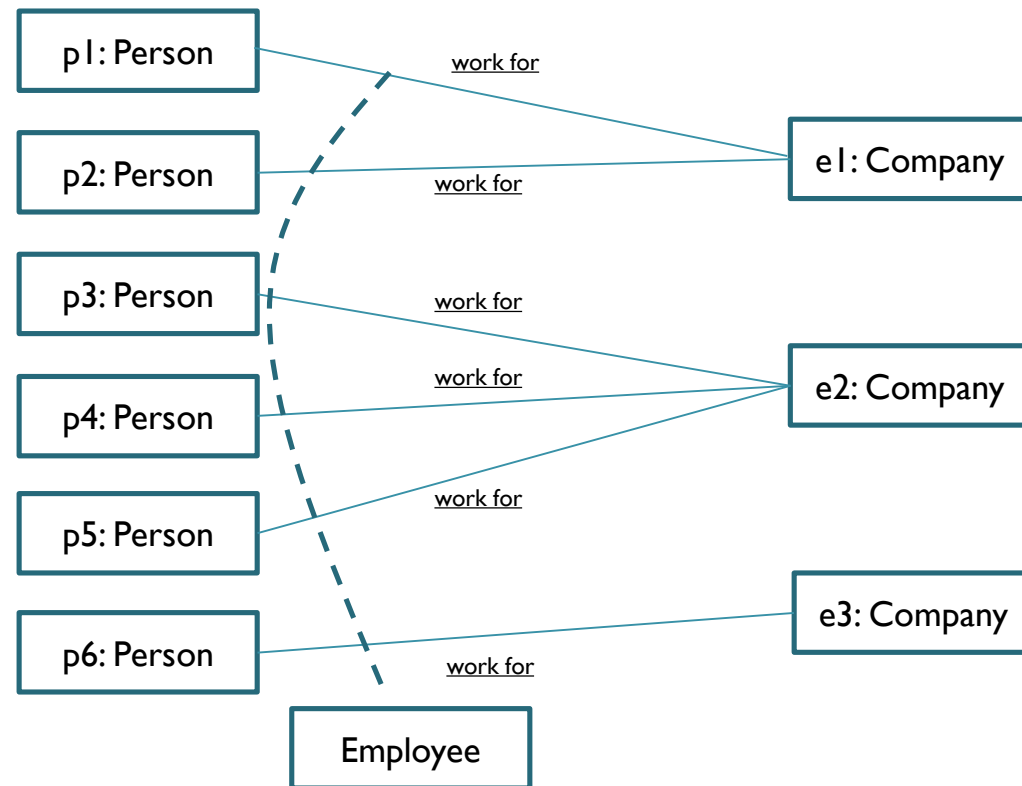


Example 1



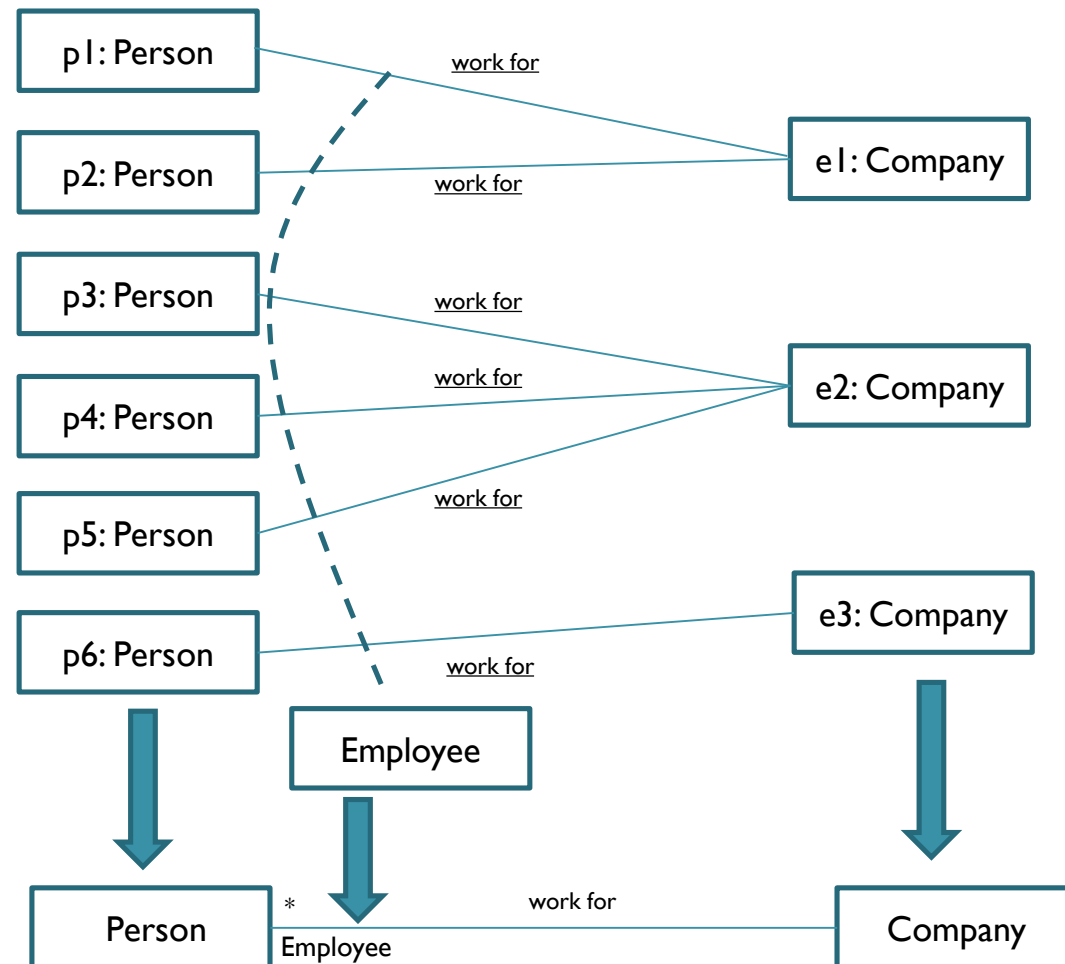
Example

I

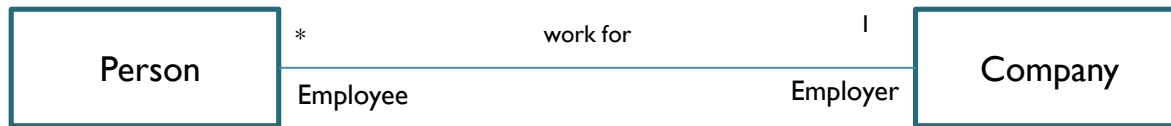


Example

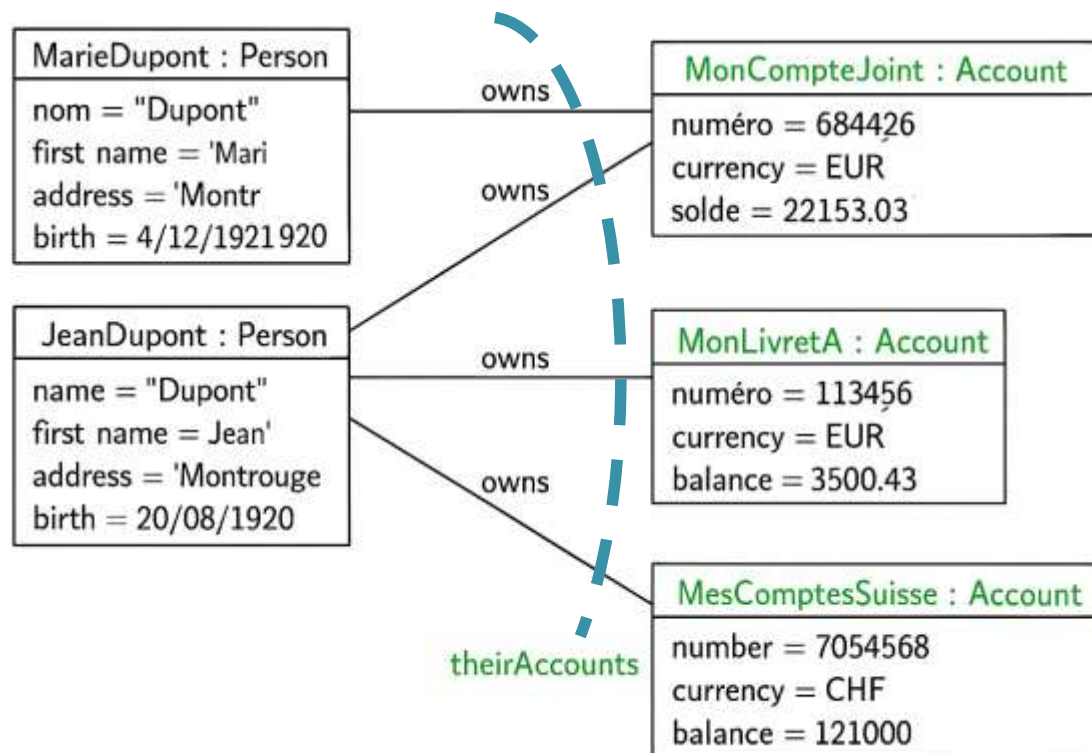
I



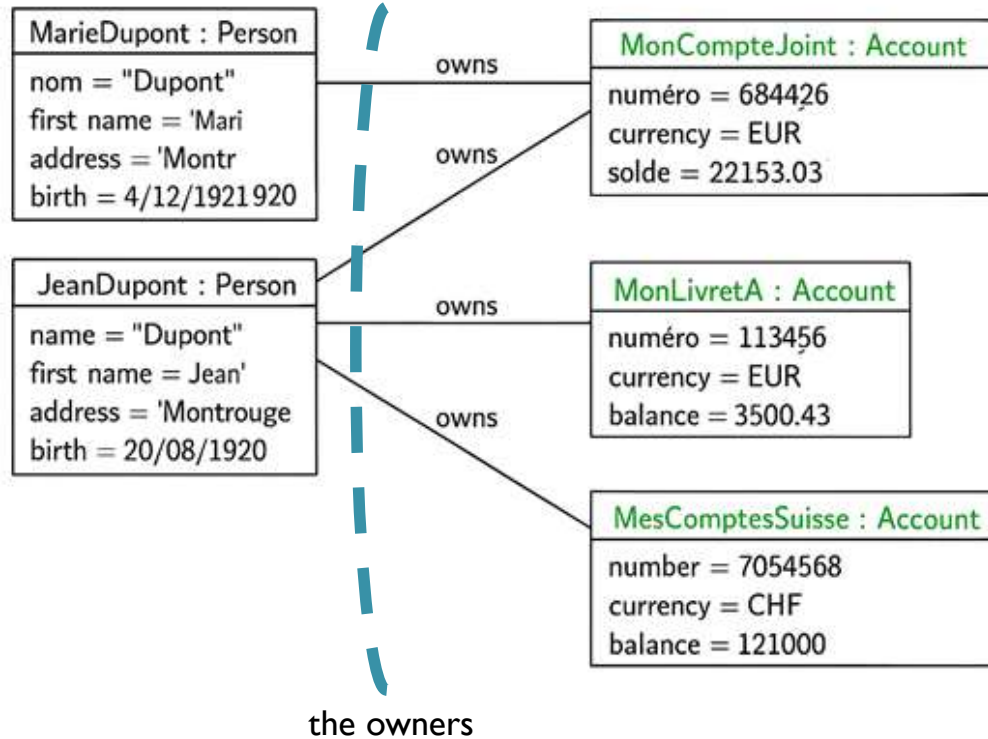
Example I



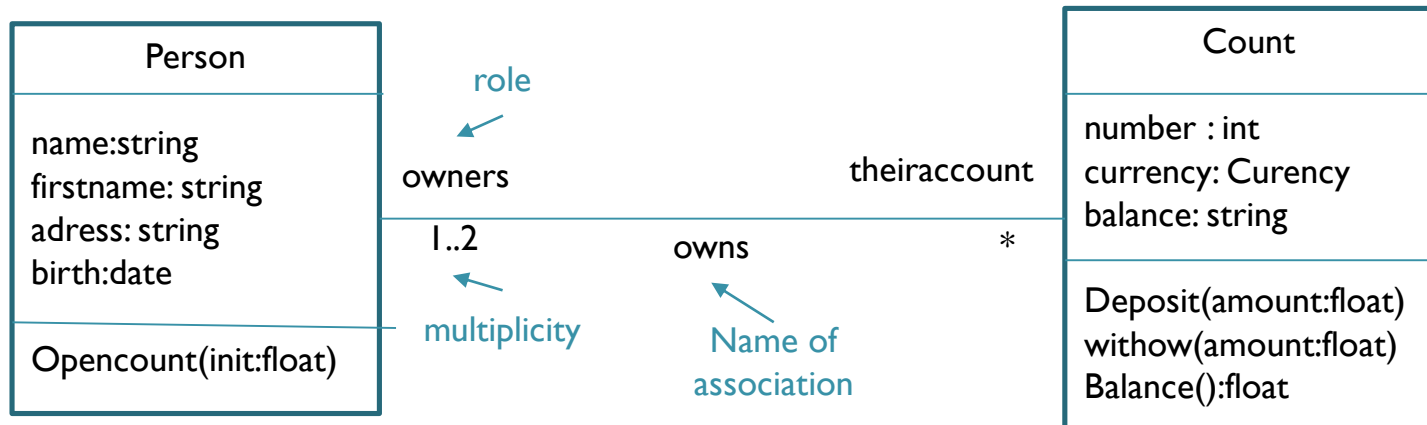
Example 2 : link between objects



Exemple 2 : lien entre objets



Example 2 : Association



Mutllicity

- Number of instances of class B associated with a single instance of class A

Exactement n



Exactement n ou m ou p



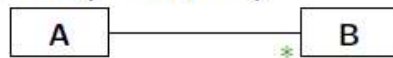
Entre n et m



Au moins n

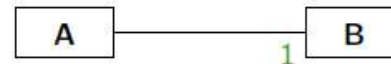


Plusieurs (0 ou plus)

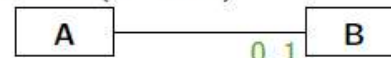


En pratique

Exactement 1



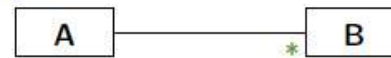
Au plus 1 (0 ou 1)



Au moins 1 (jamais 0)



0 ou plus



Example I

- A person can own several cars (between zero and an arbitrary number); a car is owned by only one person.

*

Example I

- A person can own several cars (between zero and an arbitrary number); a car is owned by only one person.

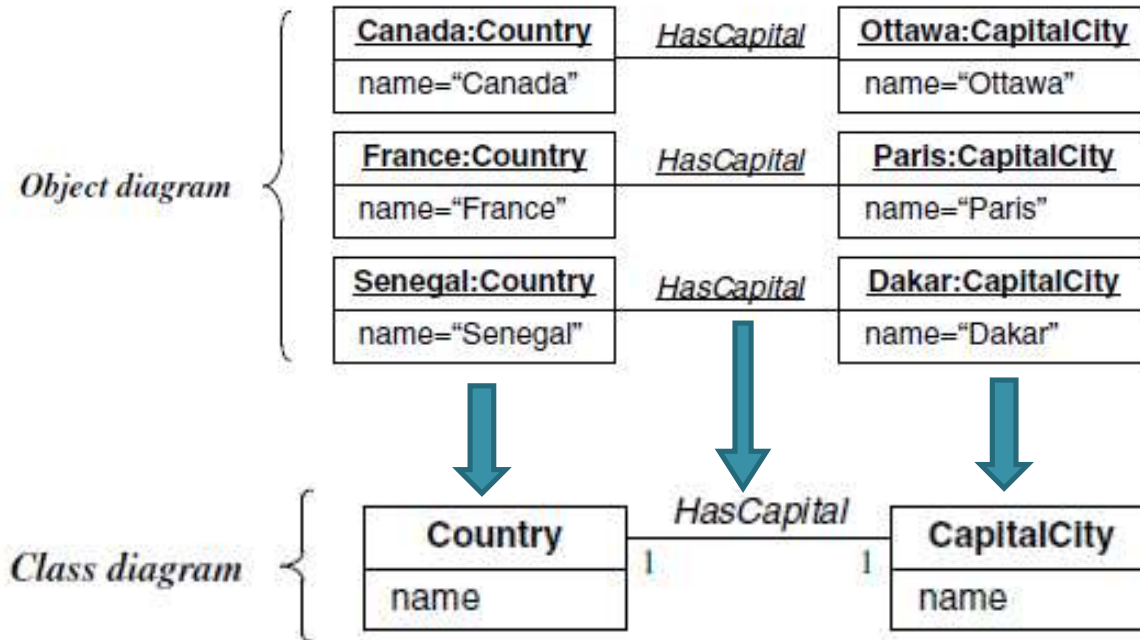


Example 2

Object diagram

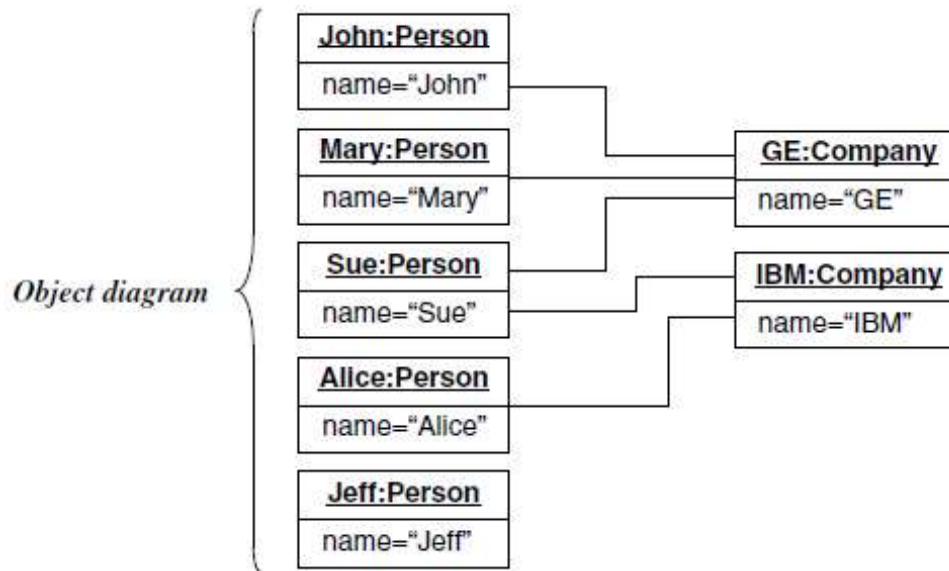


Example 2

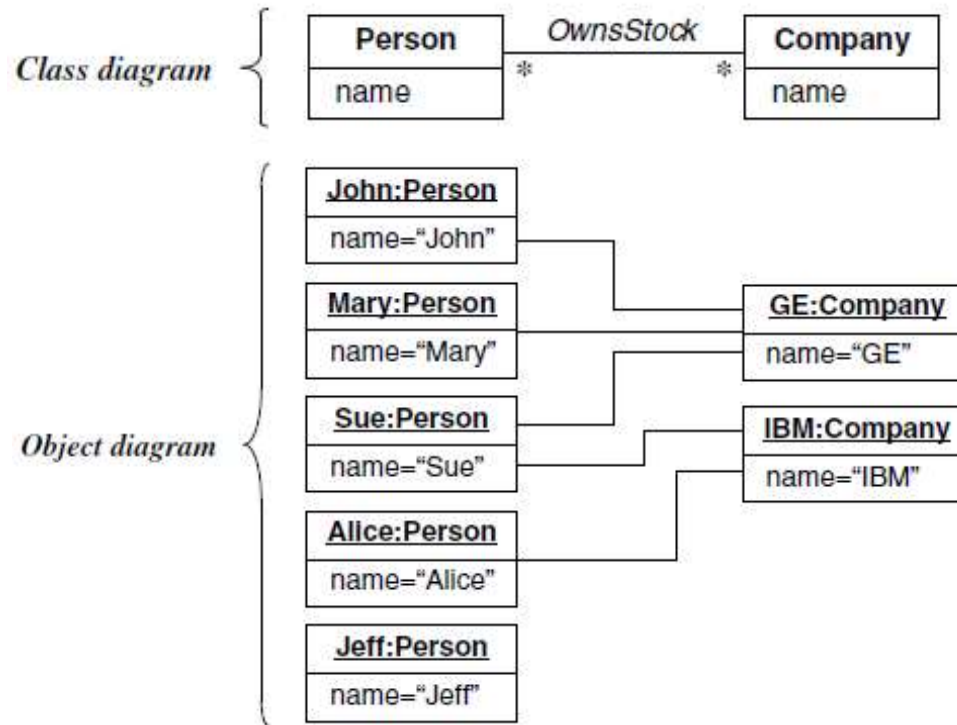


Example 3

- We assume the existence of several people who work in a company, and people can work for several companies.



Example 3



Navigability

- Navigability indicates whether it is possible to traverse an association. Navigability is graphically represented by an arrow on the navigable end, and it is prevented by a cross on the non-navigable end. By default, an association is navigable in both directions.

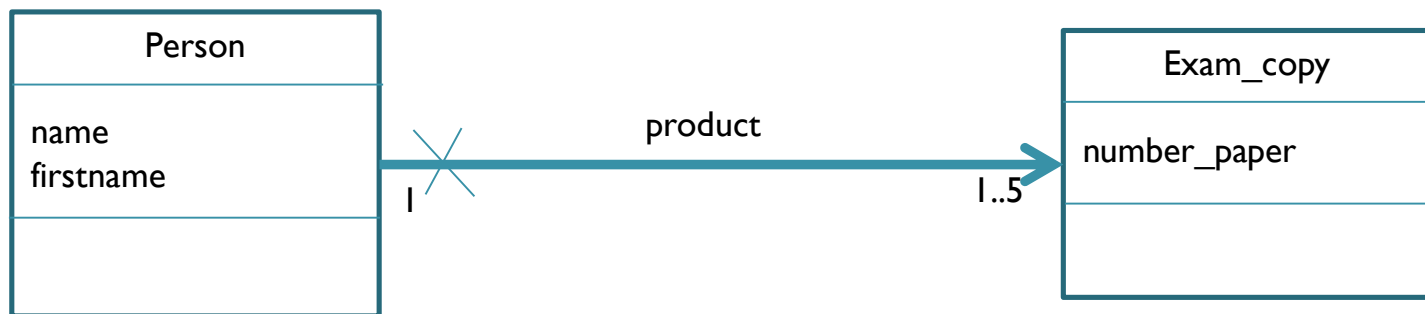


Example

- A person is associated with their exam papers, but the reverse is not possible (it is not possible to directly identify the author of an exam paper, particularly before the paper has been graded).

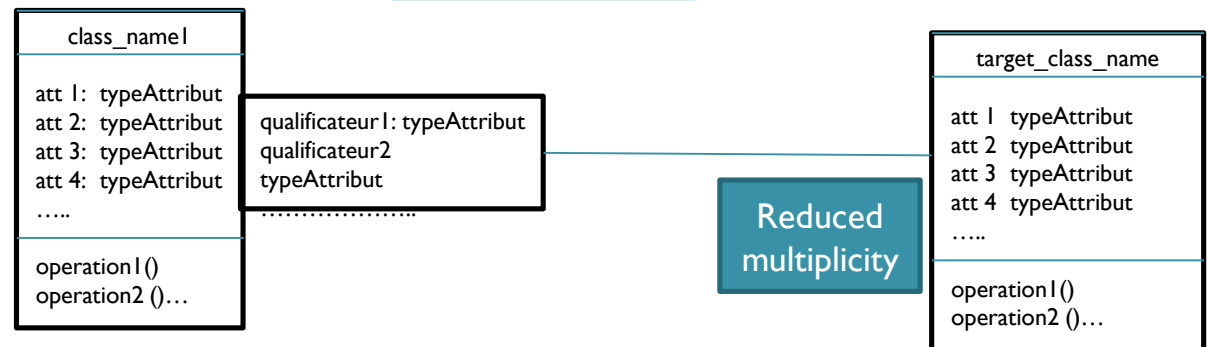
Example

- A person is associated with their exam papers, but the reverse is not possible (it is not possible to directly identify the author of an exam paper, particularly before the paper has been graded).

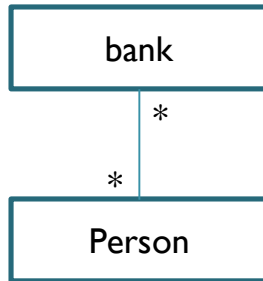


Qualified Association

- It is sometimes preferable to restrict the scope of the association to a few targeted elements (such as one or more attributes) of the class.
- These targeted elements are called a qualifier.
A qualifier thus makes it possible to select one or more objects within the set of objects of a given object (called the qualified object) that are connected through an association to another object.
- The object selected by the qualifier value is called the target object.
The association is called a qualified association.
A qualifier always applies to an association where the multiplicity is “many” (before the association is qualified) on the target side.
- A qualified object and a qualifier value generate a unique linked target object.
Considering a qualified object, each qualifier value designates a unique target object.



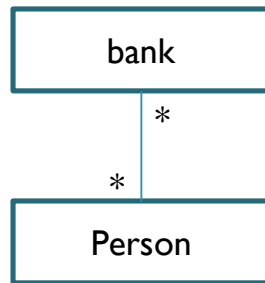
Example



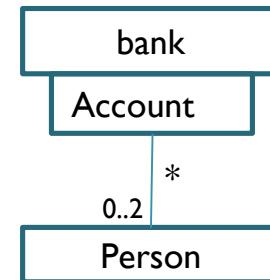
We want to restrict the number of people attached to the bank.



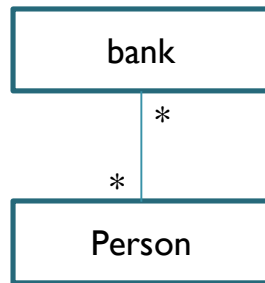
Example



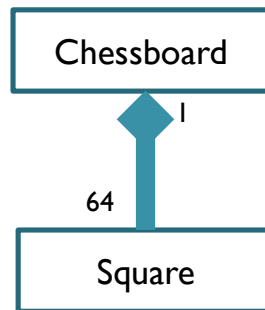
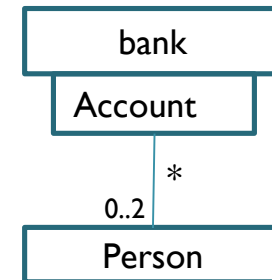
We want to restrict the number of people attached to the bank.



Exemple



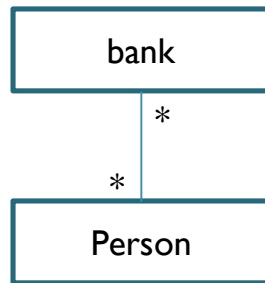
We want to restrict the number of people attached to the bank.



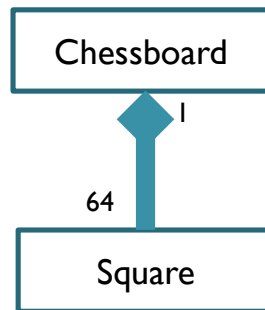
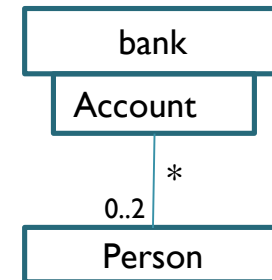
We want to restrict the number of square attached to a checkboard.



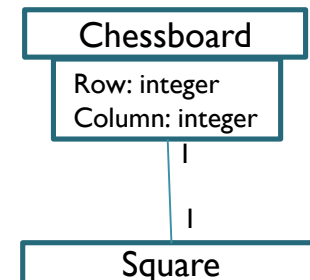
Exemple



We want to restrict the number of people attached to the bank.

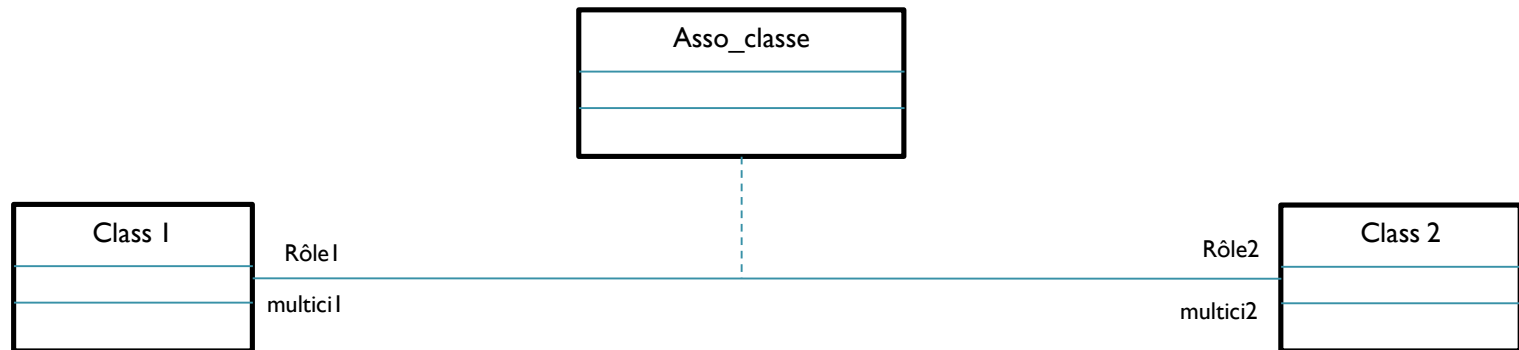


We want to restrict the number of square attached to a checkboard.



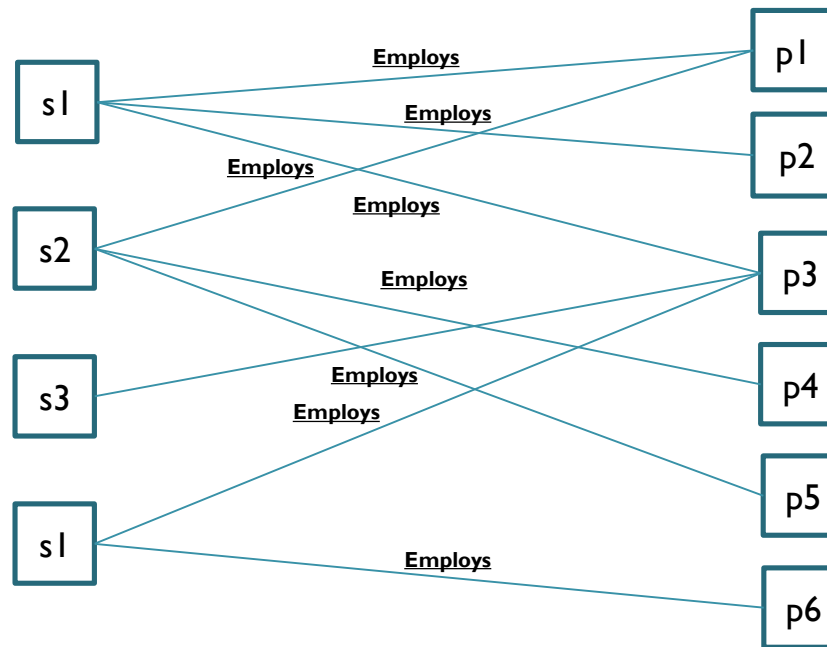
Association class

- Sometimes, an association must possess properties.
An association class combines the characteristics of both associations and classes: it connects two or more classes and also possesses attributes and operations.
An association class is represented by a dashed line between the class and the association it represents.

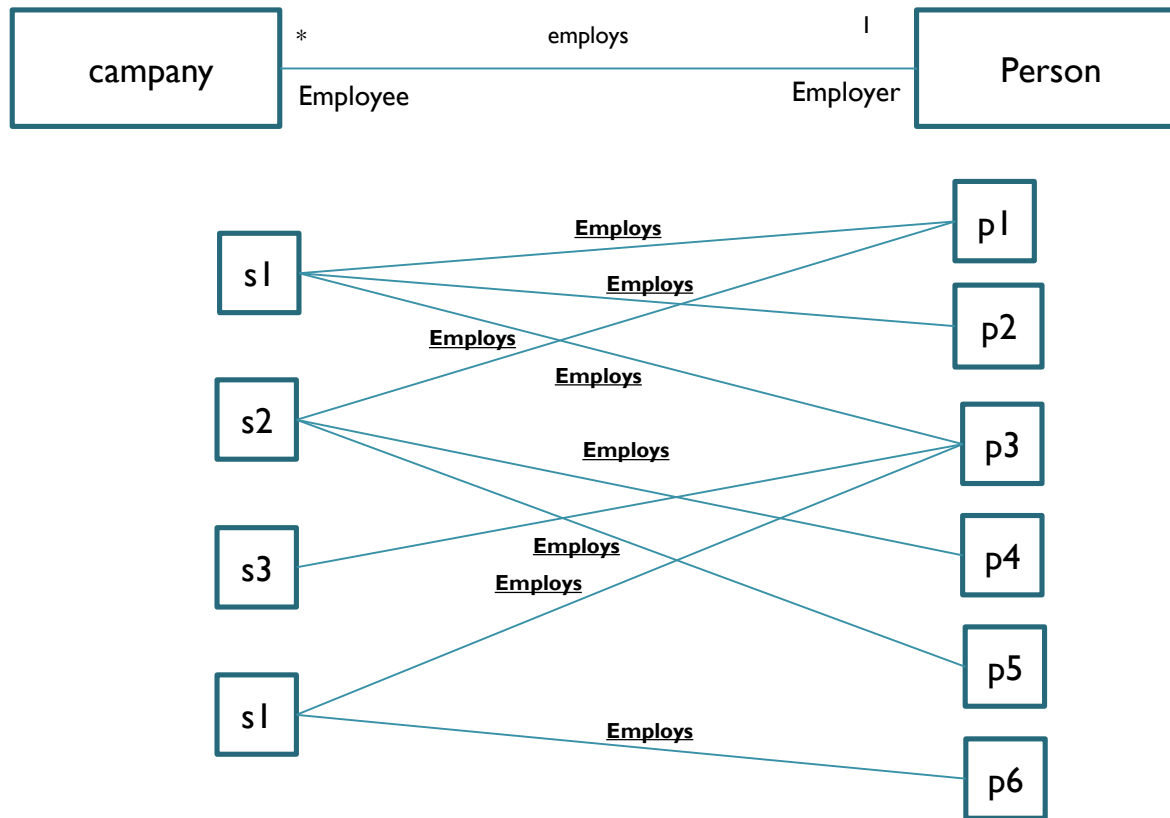


Example 1

- A company employs several people, and a person may work in multiple companies.

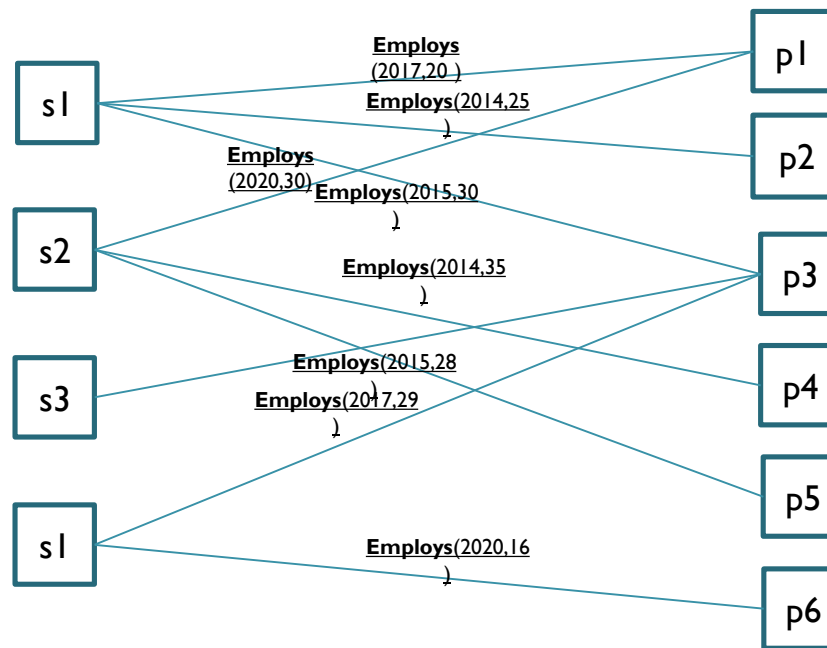


Example I

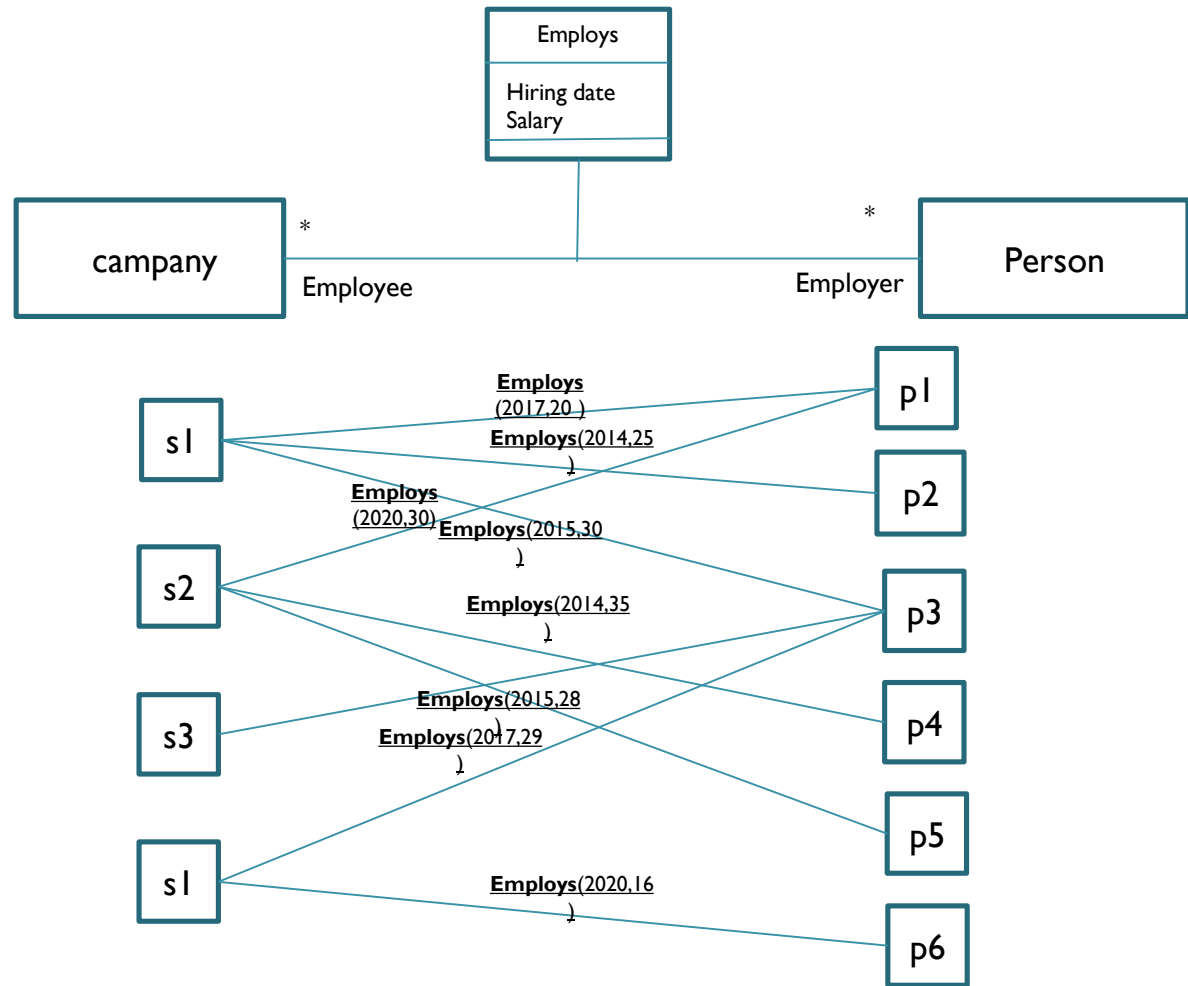


- We want to add the employee's hiring date and salary within the company.

Exemple 2

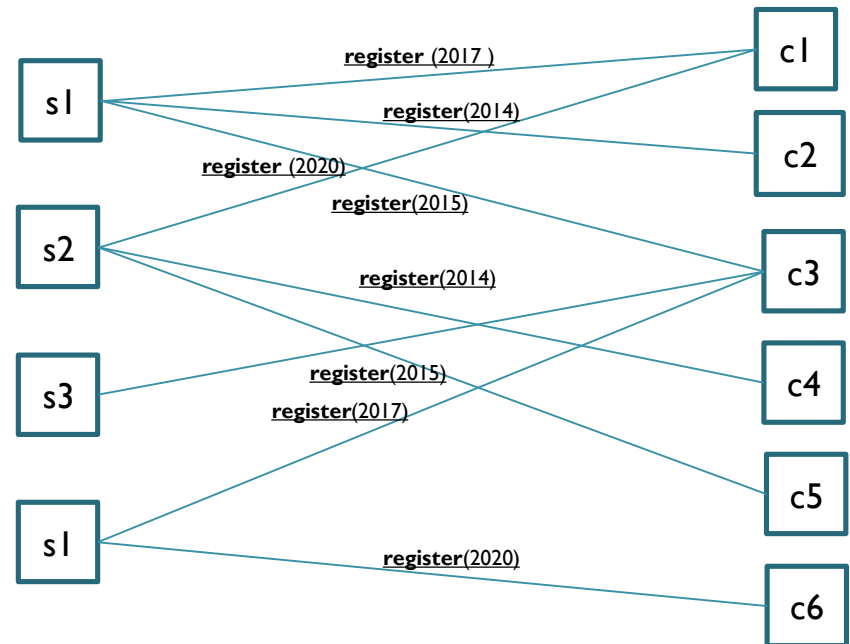


Exemple 2

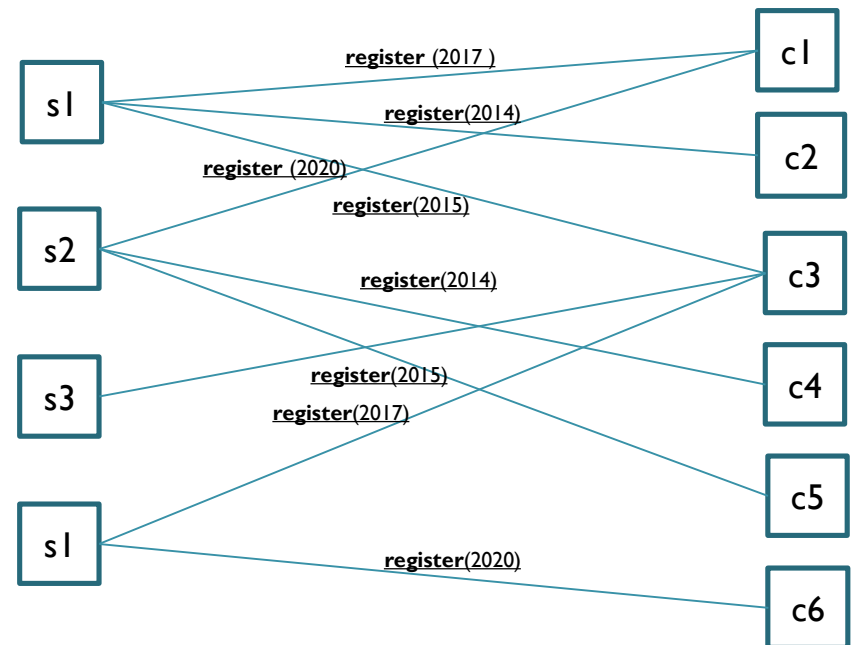
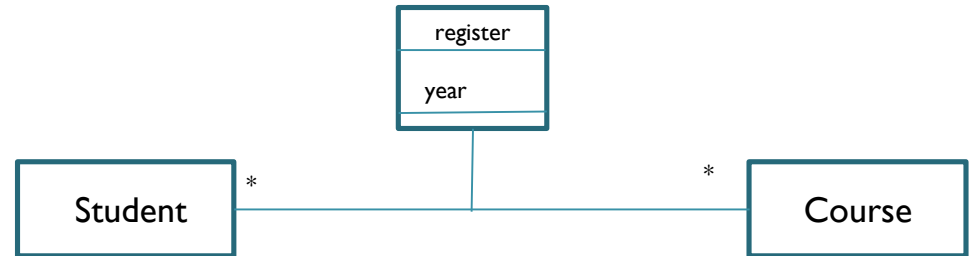


- A student can register for several courses, and in a course, several students can be registered. Each registration has a date.

Exemple 3



Example 3



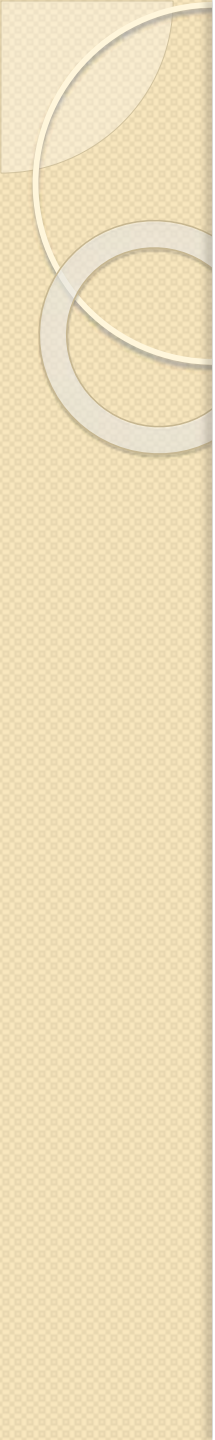
- 
- merci

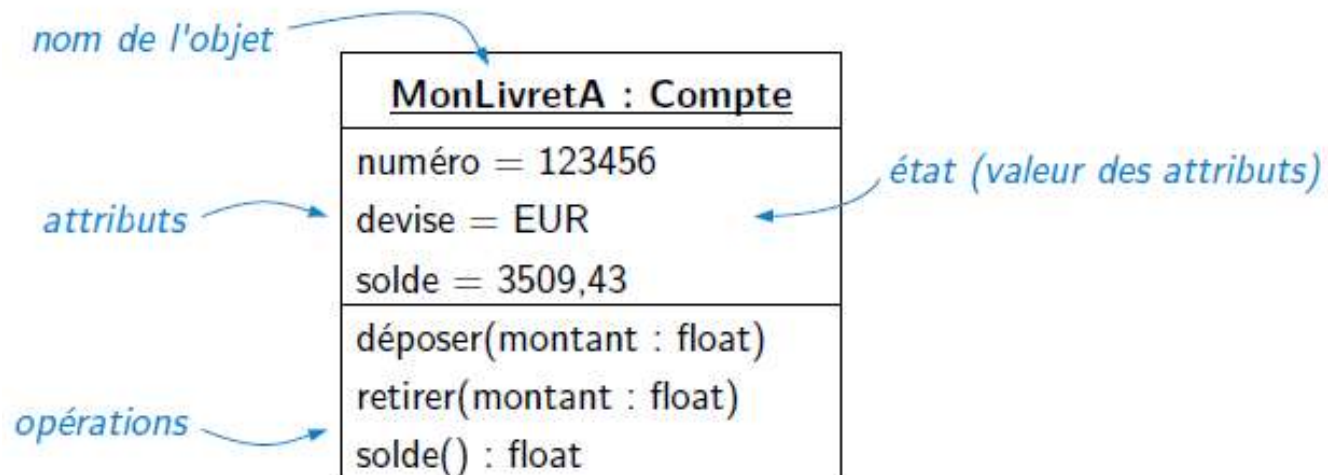
Diagramme de classe

- Le diagramme de classes est considéré comme le plus important de la modélisation orientée objet, il est le seul obligatoire lors d'une telle modélisation
- le diagramme de classes en montre la structure interne d'un système.
- Il permet de fournir une représentation abstraite des objets du système qui vont interagir pour réaliser les cas d'utilisation.
- Il s'agit d'une vue statique, car on ne tient pas compte du facteur temporel dans le comportement du système.
- le diagramme de classes permet de modéliser les classes du système et leurs relations indépendamment d'un langage de programmation particulier.

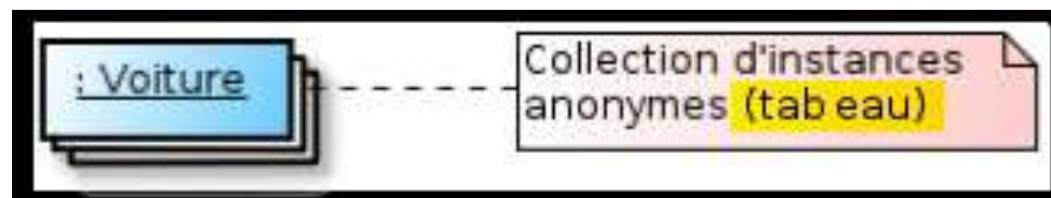
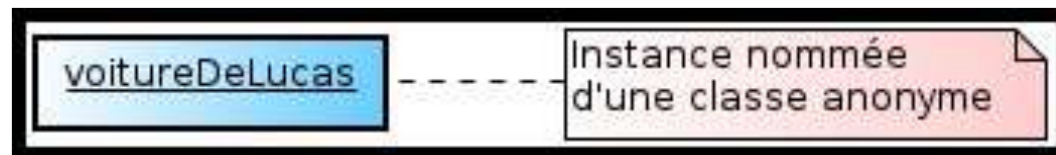
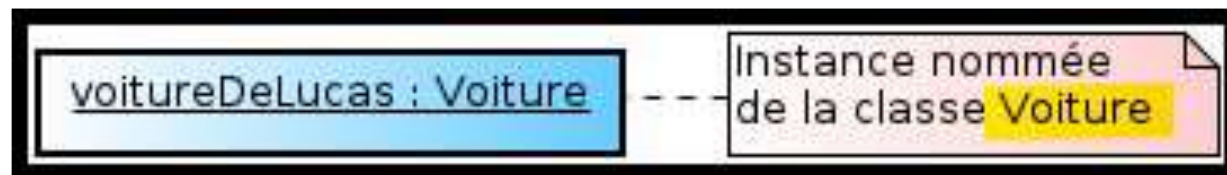
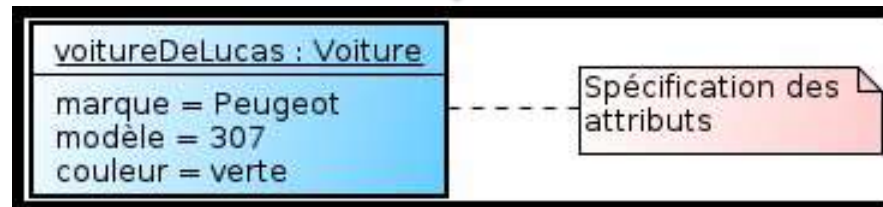
Objet

- Entité avec une limite et **une identité** bien définie, qui encapsule **un état** et un **comportement**
- Exemple 1: un objet

e3:Etudiant
Id: 11129 Nom: Salim Prénom: amir Parcours: licence Année etude: 3
CalculerMoy() Revisercours()



Exemple sur comment modéliser des objets



Objets

- Exemples d'objets

e1:Etudiant
Id: 11121 Nom: ahmed Prénom: Okba Parcours: licence Année etude: 3
CalculerMoy() Revisercours()

e3:Etudiant
Id: 11129 Nom: Salim Prénom: amir Parcours: licence Année etude: 3
CalculerMoy() Revisercours()

e2:Etudiant
Id: 11125 Nom: kamal Prénom: khaled Parcours: licence Année etude: 3
CalculerMoy() Revisercours()

e4:Etudiant
Id: 11130 Nom: Kaddour Prénom: hillal Parcours: licence Année etude: 3
CalculerMoy() Revisercours()

m1:Matiere
Id: 112 Nom: interface HM Label: IHM Parcours: licence Année etude: 3
Presenter()

m2:Matiere
Id: 122 Nom: Genie Logicie Label: GL Parcours: licence Année etude: 3
Presenter()

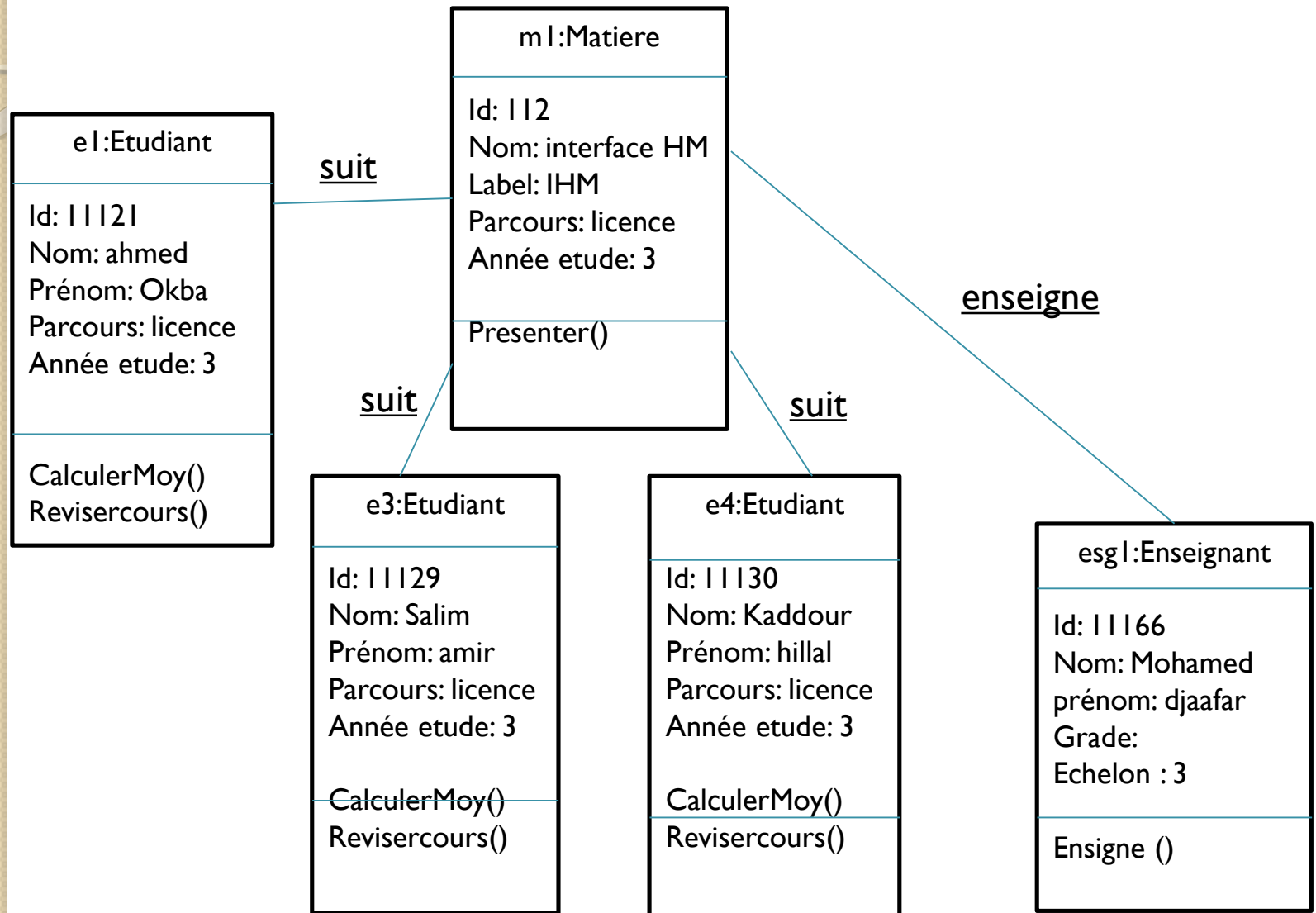
esg1:Enseignat
Id: 11166 Nom: Mohamed prénom: djaafar Grade: Echelon : 3
Ensigne ()

esg2:Enseignat
Id: 1177 Nom: Mohamed prénom: djaafar Grade: Echelon : 3
Ensigne ()

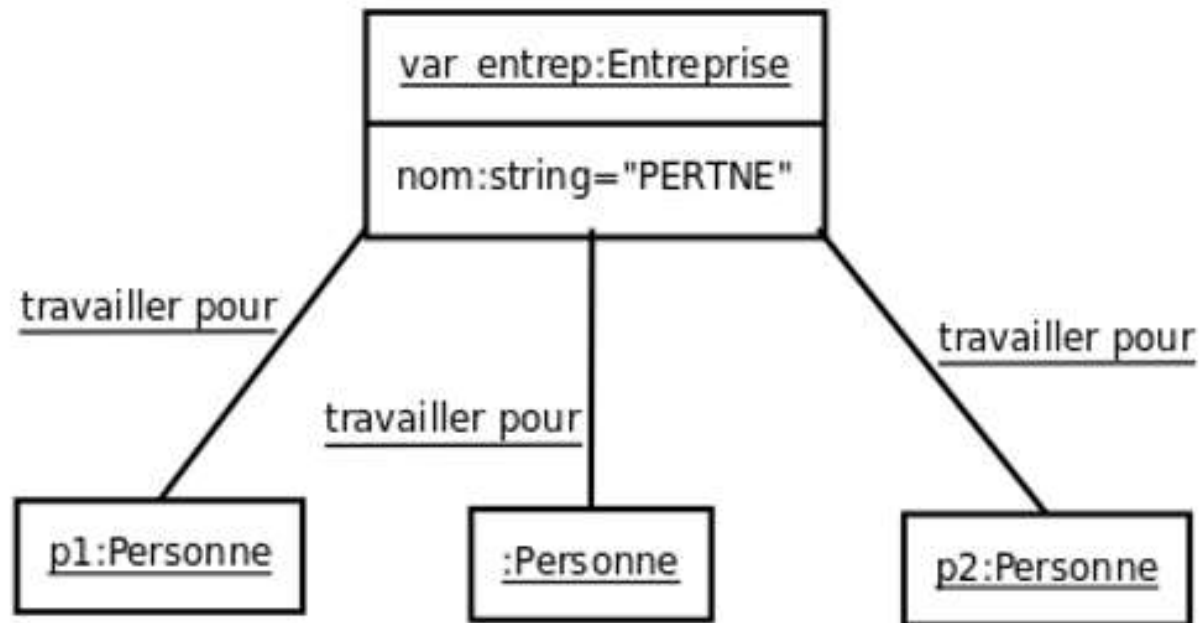
Les liens

- Les objets sont reliés par des instances d'associations : les liens.
 - Un lien représente une relation entre objets a un instant donné.
 - Un nom de lien s'il existe alors il est souligné
 - ATTENTION : la multiplicité des extrémités des liens est toujours de 1.
 - Exemple : représentation de la structure générale d'une voiture
-

Exemple I : Diagramme objet

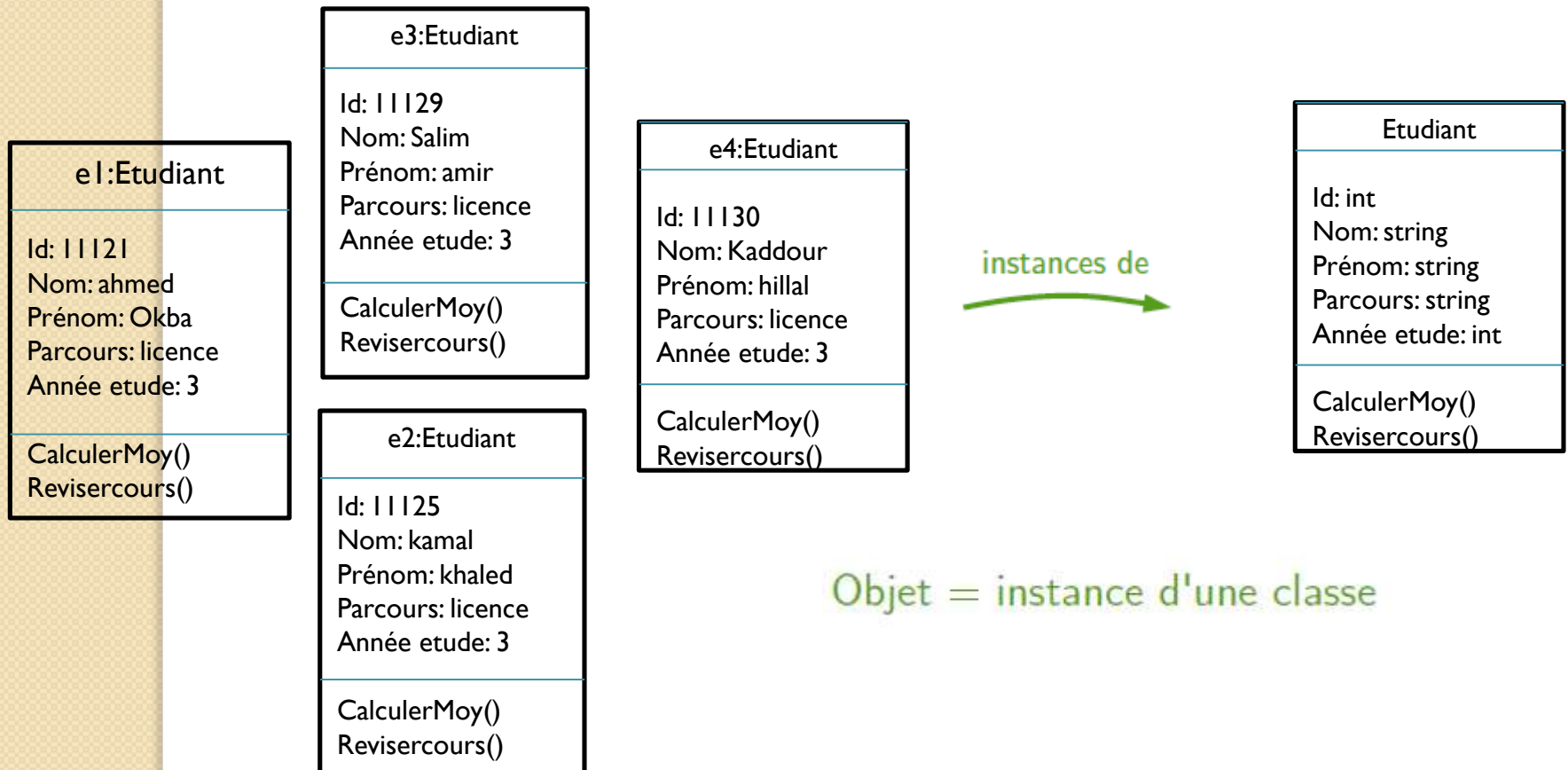


Exemple 2: Diagramme objet

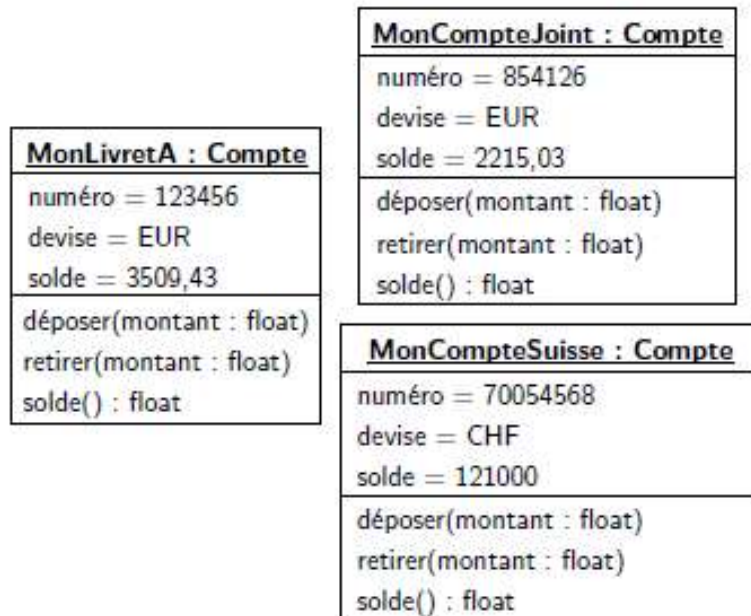


Classe

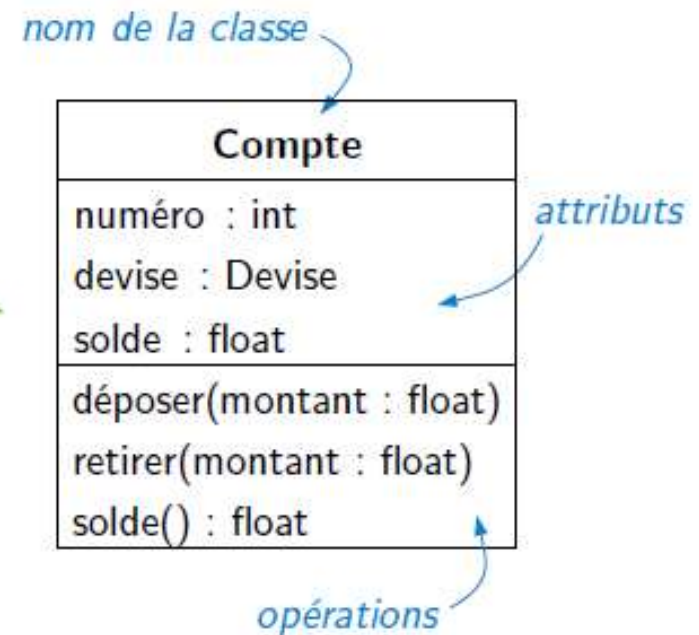
- Classe est une **description** d'un ensemble d'objets qui partagent les mêmes caractéristiques (propriétés, opérations,...) et la même sémantique



Exemple I



instances de



Une classe

- Une classe est un concept abstrait représentant des éléments variés comme :
 - des éléments concrets (ex. : des avions),
 - des éléments abstraits (ex. : des commandes de marchandises ou services),
 - des composants d'une application (ex. : les boutons des boîtes de dialogue),
 - des structures informatiques (ex. : des tables de hachage),
 - des éléments comportementaux (ex. : des tâches), etc.

Structure d'une classe

- Une Classe comporte quatre compartiments :
 - Nom (obligatoire)
 - attributs,
 - opérations,
 - Réceptions et
 - structure interne .
 - Une classe peut également avoir des compartiments facultatifs comme décrit pour les classificateurs en général
- Les compartiments étant des rectangles juxtaposé horizontalement

Nom de la classe
Attributs
Opérations
Responsabilités et/ou exception

Nom_de_la_classe
-attribut_1: type1 -attribut_2: type2
+opération_1(): type1 +opération_2(): void

Example

Window

Window

size: Area
visibility: Boolean
display()
hide()

Window

attributes

+size: Area = (100, 100)
#visibility: Boolean = true
+defaultSize: Rectangle
-xWin: XWindow

operations

display()
hide()
-attachX(xWin: XWindow)

Window

attributes

public
size: Area = (100, 100)
defaultSize: Rectangle
protected
visibility: Boolean = true
private
xWin: XWindow

operations

public
display()
hide()
private
attachX(xWin: XWindow)

Car

internal structure



Nom de classe

- Le nom de la classe doit évoquer le concept décrit par la classe.
- Il commence par **une majuscule**.
- On peut ajouter des informations subsidiaires comme le nom de l'auteur de la modélisation, la date, etc.
- Pour indiquer qu'une classe est abstraite, il faut ajouter le mot-clef **abstract**.
- La syntaxe de base de la déclaration d'un nom d'une classe est la suivante

[<Nom_du_paquetage_1>::...::<Nom_du_paquetage_N>] <Nom_de_la_classe>

[{ [abstract], [<auteur>], [<date>], ... }]

Exemples

Classe Roue,

Roue

Classe roue contenu dans
package Véhicule

Vehecule :: Roue

La classe roue avec des
informations

Roue {abstract, auteur::ahmed }

Attributs (propriété)

- Propriété nommée d'un classificateur qui décrit une plage de valeur que les instance d'un classificateur peuvent prendre
- Le nom d'un attribut commence avec une minuscule
- Exemple:
- Le nom, prénom, adresse, mail, numéro.... Sont les propriétés nommées de la classe personne

Exemple attributs

nom	attribut
Chien	Nom, race, âge, couleur
Compte	N°, type, solde, ligne de crédit
Téléphone	N°, marque, sonnerie, répertoire, opérateur
Voiture	Plaque, marque, couleur, vitesse

attributs

- La forme générale:
- $\langle \textit{property} \rangle ::=$
- $[\langle \textit{visibility} \rangle] ["/"] \textbf{\langle name \rangle} [":" \langle \textit{prop-type} \rangle] [[" \langle \textit{multiplicity-range} \rangle "] ["=" \langle \textit{default} \rangle] [{" \langle \textit{prop-modifier} \rangle [" \langle \textit{prop-modifier} \rangle]^* "}]]$

[<visibility>] ['/' <name> [':' <prop-type>] ['[' <multiplicity-range> ']] ['=' <default>] ['{' <prop-modifier> [',' <prop-modifier>]* '}']

- <visibility> : est la visibilité de la propriété.
<visibilité> ::= '+' | '-' | '#' | '~'
- '/' signifie que la propriété est dérivée.
- <nom> est le nom de la Propriété,
- <prop-type> est le nom du classificateur qui est le type de la propriété.
- <multiplicity-range> est la plage de multiplicité de la propriété. Si ce terme est omis, il implique une multiplicité de 1 (exactement un).
- <default> est une expression qui correspond à la ou aux valeurs par défaut de la propriété.
- <prop-modifier> indique un modificateur qui s'applique à la propriété.
- <prop-modifier> ::= 'readOnly' | 'union' | 'subsets' <property-name> | 'redefines' <property-name> | 'ordered' | 'unordered' | 'unique' | 'nonunique' | 'seq' | 'sequence' | 'id' | <prop-constraint>

Example

- name: String
- shape: Rectangle
- +size: Integer[2..5]
- /area: Integer[5] {readOnly}
- -height: Integer = 5
- width: Integer

Type primitive

- Integer : Une instance de Integer est une valeur dans l'ensemble (infini) d'entiers
- Boolean : Une instance de Boolean est l'une des valeurs prédéfinies true et false.
- String : Une instance de String définit une séquence de caractères.
- UnlimitedNatural: La valeur illimitée est indiquée à l'aide d'un astérisque (« * »).
- Real(float): Une instance de Real est une valeur dans l'ensemble (infini) des nombres réels.

<prop-modifier>

- **'readOnly'** signifie que la propriété est en lecture seule.
- **'union'** signifie que la Propriété est une union dérivée de ses sous-ensembles.
- **'subsets'** <property-name> signifie que la Propriété est un sous-ensemble approprié de la Propriété identifiée par <property-name>, où <property-name> peut être qualifié.
- **'redefines'** <property-name> signifie que la Propriété redéfinit une Propriété héritée identifiée par <property-name>, où <property-name> peut être qualifié.
- **'ordered'** signifie que la propriété est ordonnée,
- **'unordered'** signifie que la propriété n'est pas ordonnée,
- **'unique'** signifie qu'il n'y a pas de doublons dans une propriété à plusieurs valeurs,
- **"non unique"** signifie qu'il peut y avoir des doublons dans une propriété à plusieurs valeurs,
- **'seq' ou 'sequence'** signifie que la propriété représente un sac ordonné,
- **'id'** signifie que la propriété fait partie de l'identifiant de la classe.
- **<prop-constraint>** est une expression qui spécifie une contrainte qui s'applique à la propriété.

Attributs dérivés

- Les attributs dérivés peuvent être calculés à partir d'autres attributs et de formules de calcul.
- Lors de la conception, un attribut dérivé peut être utilisé comme marqueur jusqu'à ce que vous puissiez déterminer les règles à lui appliquer.
- Les attributs dérivés sont symbolisés par l'ajout d'un « / » devant leur nom.

Opération

- Service qui peut être demandé à tout objet de la classe
- Comportement commun à tous les objets de la classe
 - méthode = implantation de l'opération
- Exemple d'opérations

nom	attribut	opération
Chien	Nom, race, âge, couleur	Aboier, chercher le baton, mordre, faire le beau
Compte	N°, type, solde, ligne de crédit	Retrait, virement, dépôt, consultation du solde
Téléphone	N°, marque, sonnerie, répertoire, opérateur	Appeler, Prendre un appel, Envoyer SMS, Charger
Voiture	Plaque, marque, couleur, vitesse	Tourner, accélérer, s'arrêter, faire le plein, klaxonner

Operation

- La syntaxes général
- [*<visibility>*] *<name>* ‘(‘ [*<parameter-list>*] ‘)’ [*‘:*’
[*<return-type>*] [*‘[‘ <multiplicity-range> ‘]*’] [*‘{‘*
<oper-property> [‘,’ <oper-property>] }’]*’]

[<visibility>] <name> '(' [<parameter-list>] ')' [':' [<return-type>] ['[' <multiplicity-range> ']] ['{' <oper-property> ['<oper-property>']* '}]']

- **<visibility>** est la visibilité de l'opération .
- **<visibilité>** ::= '+' | '-' | '#' | '~'
- **<nom>** est le nom de l'opération.
- **<parameter-list>** est une liste de paramètres de l'opération au format suivant :

<liste-paramètres> ::= <paramètre> [','<paramètre>]* où
<paramètre>

- **<return-type>** est le type du paramètre de résultat de retour si l'opération en a défini un.
- **<multiplicity-range>** est la multiplicité du type de retour
- **<oper-property>** indique les propriétés de l'opération.
 - **<oper-property>** ::= 'redéfinit' <oper-name> | 'requête' | 'commandé' | 'non ordonné' | 'unique' | 'non unique' | 'suite' | 'séquence' | <oper-contrainte>

Paramètre dans une operation

- $\langle \text{paramètre} \rangle ::= [\langle \text{direction} \rangle] \langle \text{parameter-name} \rangle ':' \langle \text{type-expression} \rangle ['[' \langle \text{multiplicity-range} \rangle ']'] ['=' \langle \text{default} \rangle] [\{ ' \langle \text{parm-property} \rangle [',' \langle \text{parm-property} \rangle]^* ' \}]$
- ou :
 - $\langle \text{direction} \rangle ::= \text{'in' | 'out' | 'inout'}$ (defaults to 'in' if omitted).
 - $\langle \text{parameter-name} \rangle$ le nom du parameter.
 - $\langle \text{type-expression} \rangle$ est une expression qui spécifie le type du paramètre
 - $\langle \text{multiplicity-range} \rangle$ est la multiplicité du Paramètre.
 - $\langle \text{default} \rangle$ est une expression qui définit la spécification de valeur pour la valeur par défaut du paramètre.
 - $\langle \text{parm-property} \rangle$ indique des valeurs de propriété supplémentaires qui s'appliquent au paramètre

Exemple

- display ()
- -hide ()
- +createWindow (location: Coordinates, container: Container [0..1]):Window
- +toString (): String
- +calculer(in a: integer, in b: boolean [5], out c: Real) : Real

Propriété d'un paramètre

- $\langle \textit{parm-property} \rangle ::= \text{'ordered'} \mid \text{'unordered'} \mid \text{'unique'}$ ou:
- 'ordered' s'applique lorsqu'il existe un paramètre à valeurs multiples et signifie que ses valeurs sont ordonnées.
- 'unordonné' s'applique lorsqu'il existe un paramètre à valeurs multiples et signifie que ses valeurs ne sont pas ordonnées.
- 'unique' s'applique lorsqu'il existe un paramètre à plusieurs valeurs et signifie que ses valeurs n'ont pas de doublons.

oper-property

- '**redefines**' <oper-name> signifie que l'Opération redéfinit une Opération héritée identifiée par <opername>, où <oper-name> peut être qualifié.
- « **requête** » signifie que l'opération ne modifie pas l'état du système.
- « **ordonné** » s'applique lorsqu'il existe un paramètre de retour à valeurs multiples et signifie que ses valeurs sont ordonnées.
- '**unordered**' s'applique lorsqu'il y a un paramètre de retour à plusieurs valeurs et signifie que ses valeurs ne sont pas commandé.
- '**unique**' s'applique lorsqu'il y a un paramètre de retour à plusieurs valeurs et signifie que ses valeurs n'ont pas doublons.
- "**non unique**" s'applique lorsqu'il existe un paramètre de retour à valeurs multiples et signifie que ses valeurs peuvent avoir doublons.
- '**seq**' ou '**sequence**' s'applique lorsqu'il y a un paramètre de retour à plusieurs valeurs et signifie que ses valeurs constituent un sac ordonné,
- <**oper-constraint**> est une contrainte qui s'applique à l'opération. La liste des paramètres peut être supprimée.

Des classes

Horloge
- heures : int - minutes : int
+ régler() : void + afficher() : void

LecteurMP3
- volume : int - sélection : int
+ mettreEnMarche() : void + arrêter() : void + lireSélection() : void + avanceRapide() : void + retourRapide() : void + changerVolume() : void + morceauSuivant() : void + morceauPrécédent() : void

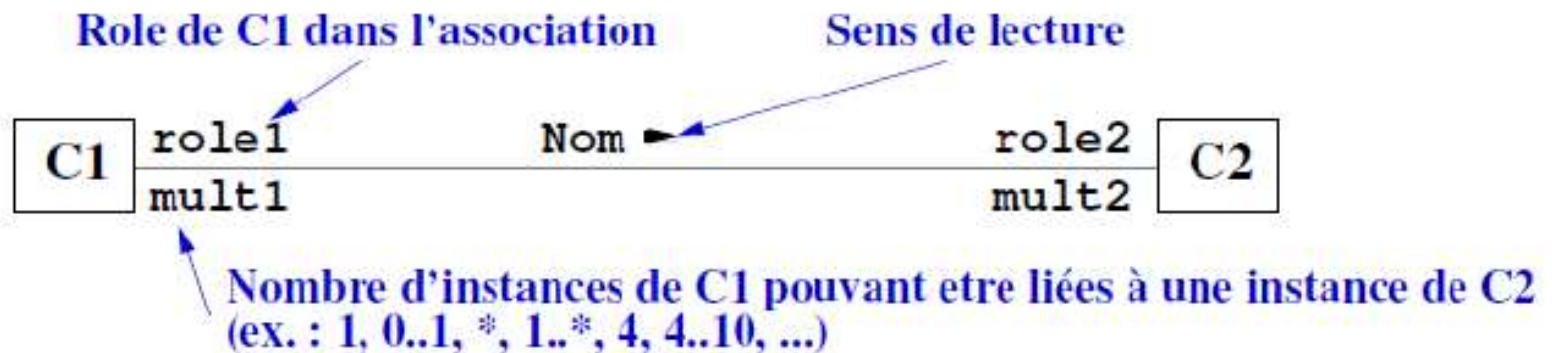
Rectangle
Longueur Largeur
Surface()

Relation entre classe

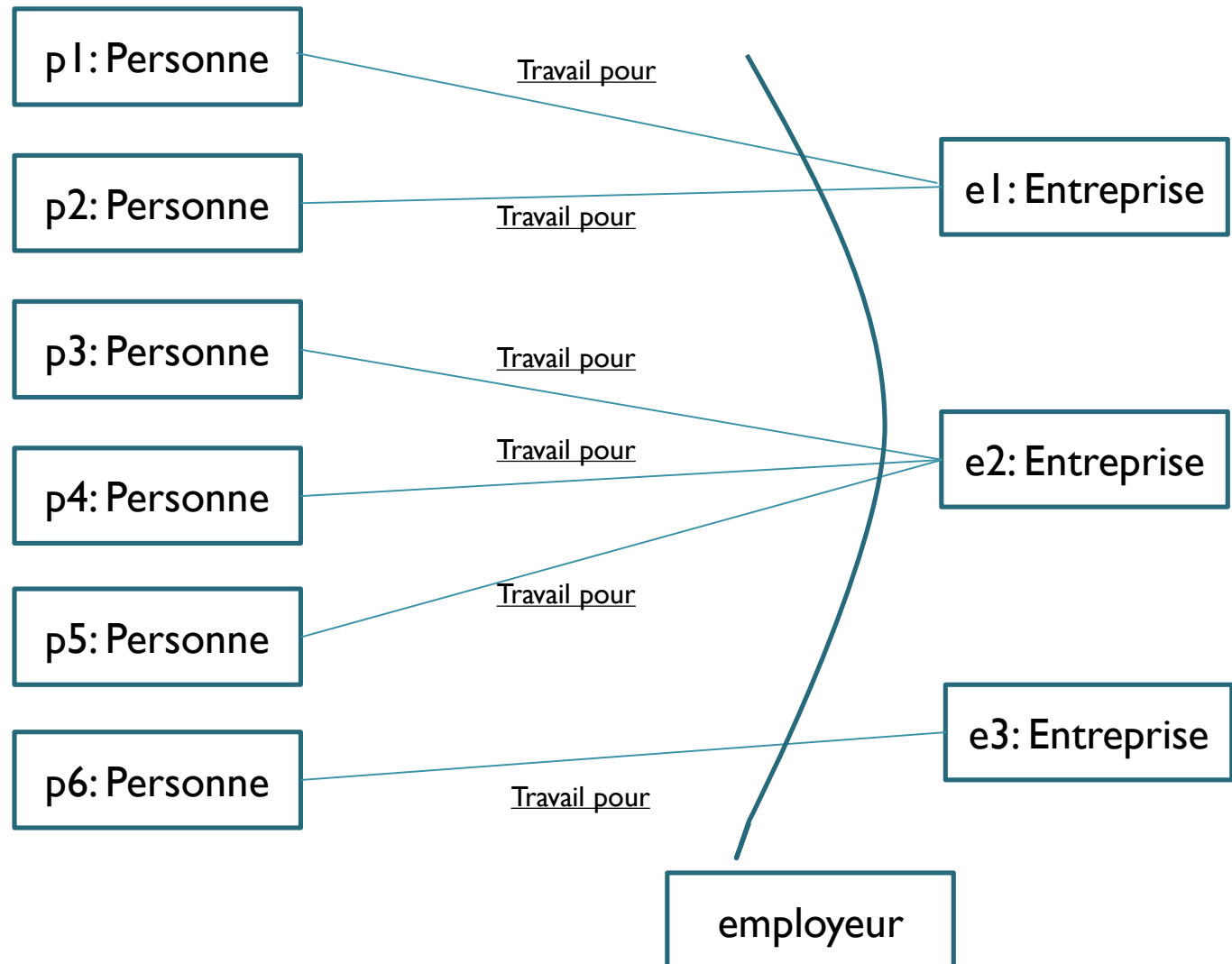
- Association (binaire et n-aire)
 - Agrégation
 - Composite
- Généralisation
- Dépendance

Association

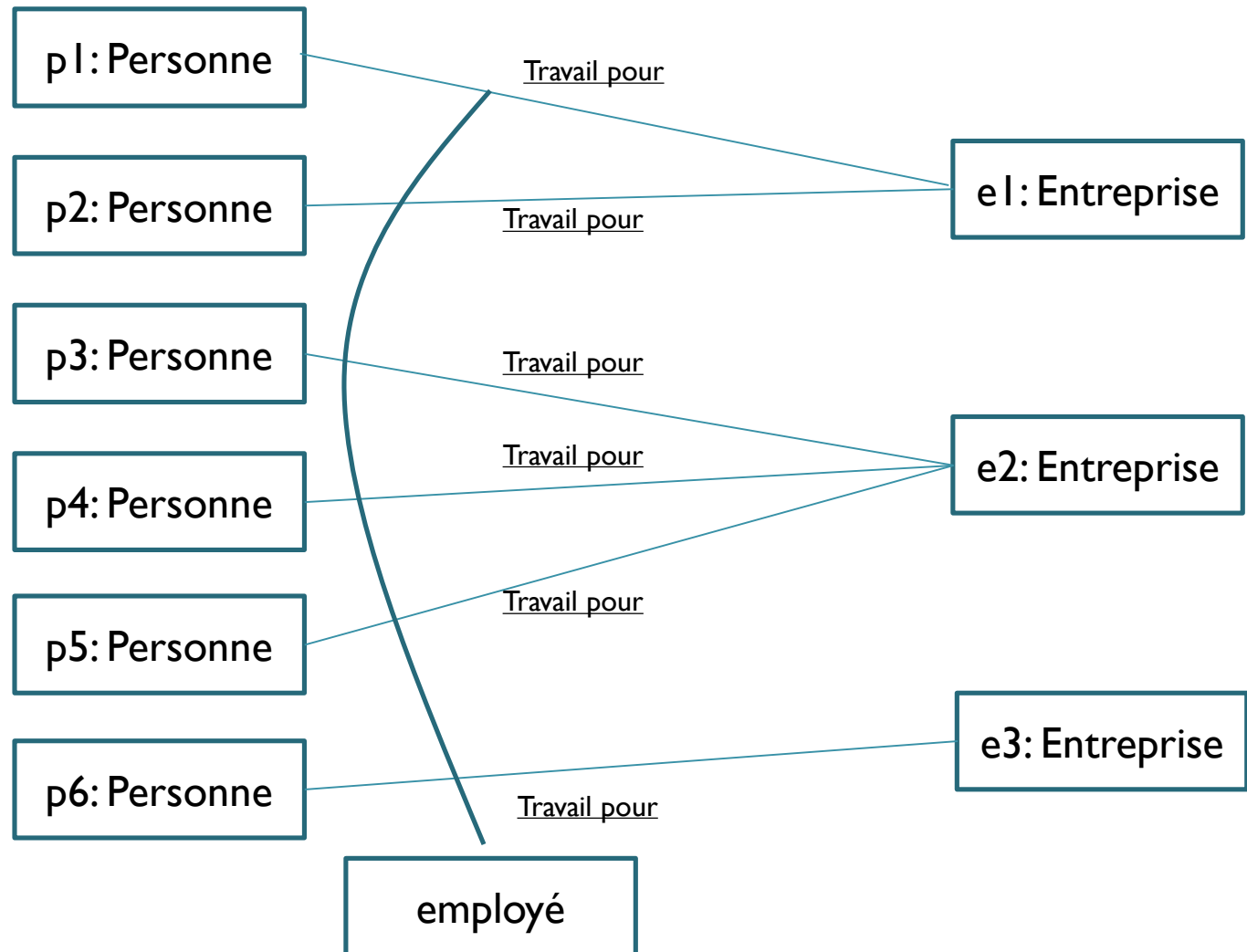
- Relation sémantique entre deux classificateurs ou plus, qui implique des connexions entre leurs instances
- En représente une association a travers un trait continu
- Chaque association possède
 - Nom : forme verbale
 - Les rôles : forme nominale décrivant le statut de la classe dans l'association. Peut contenir un mode d'accès.
 - multiplicité: nombre (ou intervalle de nombres) d'instance(s) que l'association peut impliquer



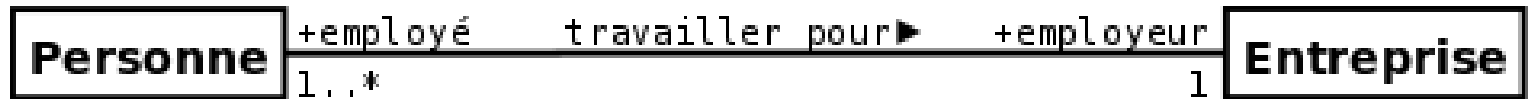
Exemple I



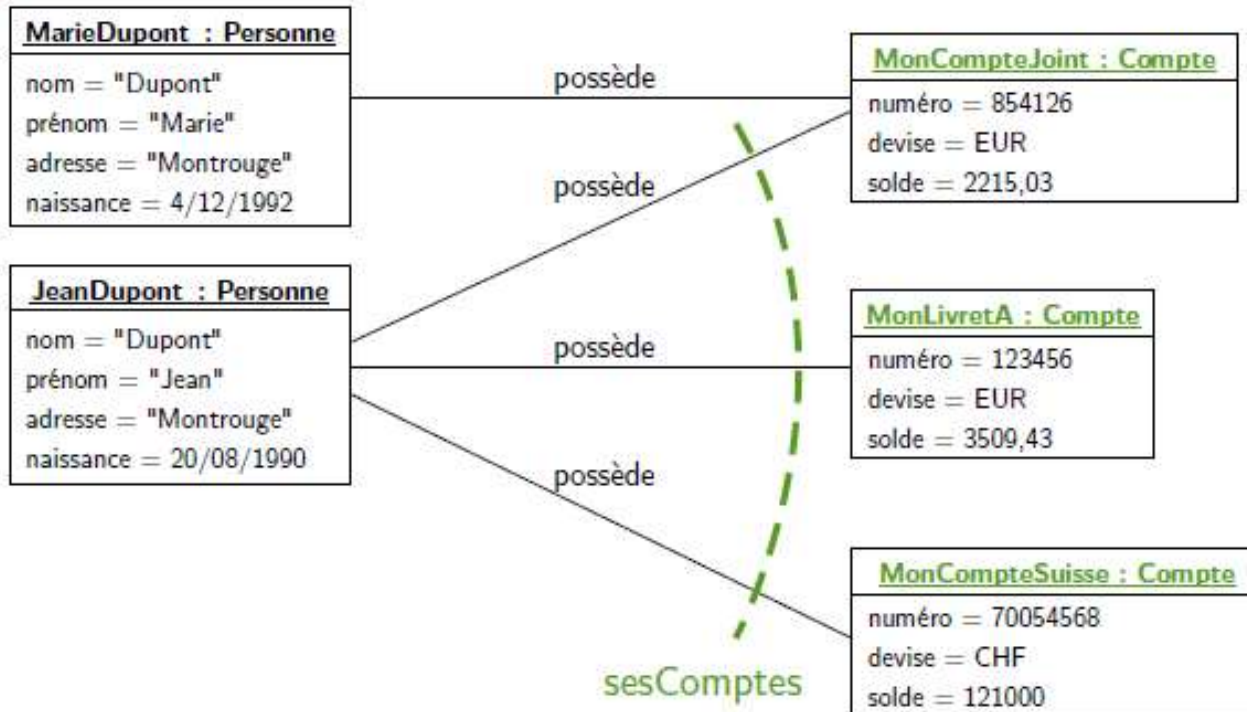
Exemple I



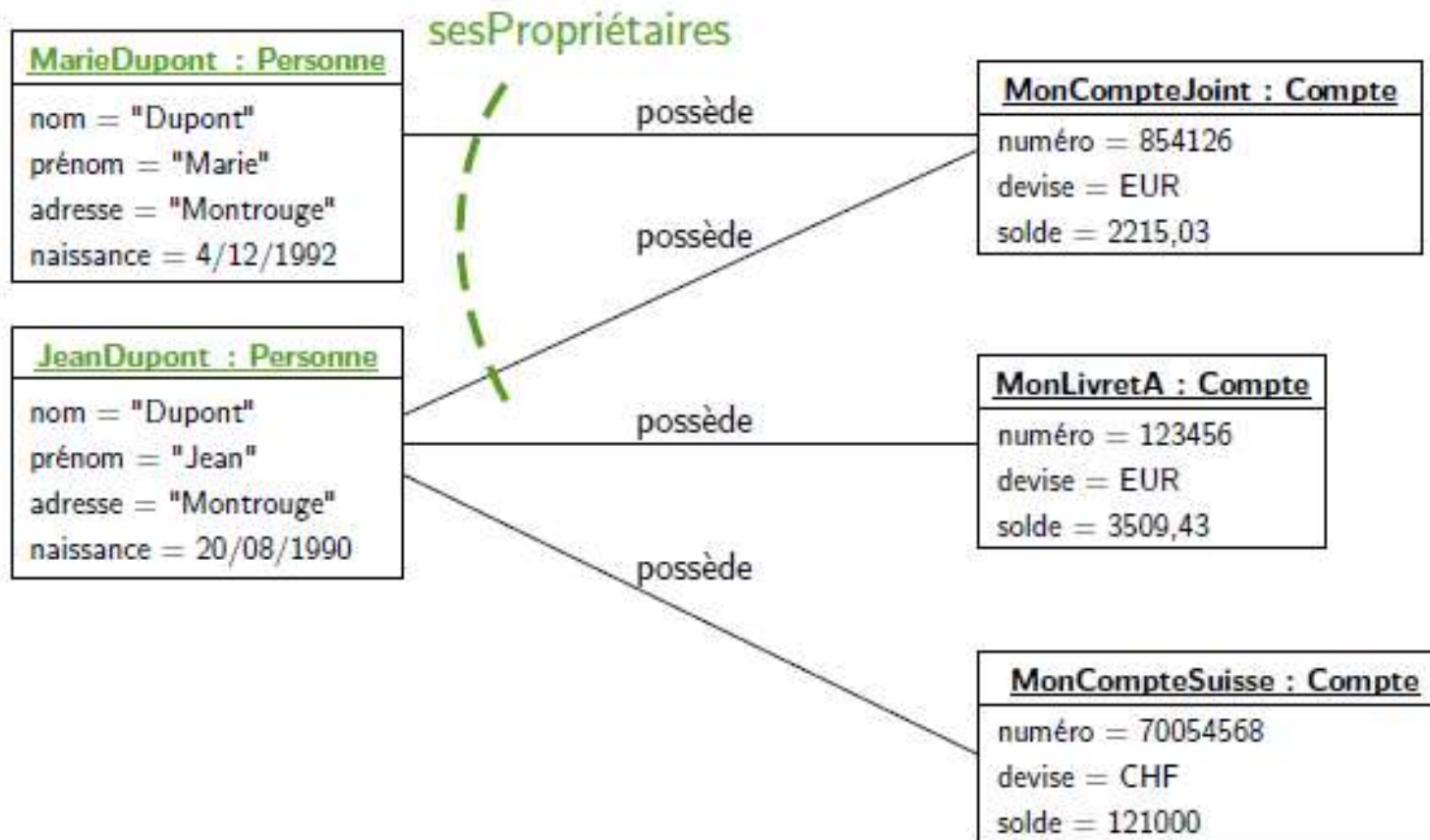
Exemple I (Association)



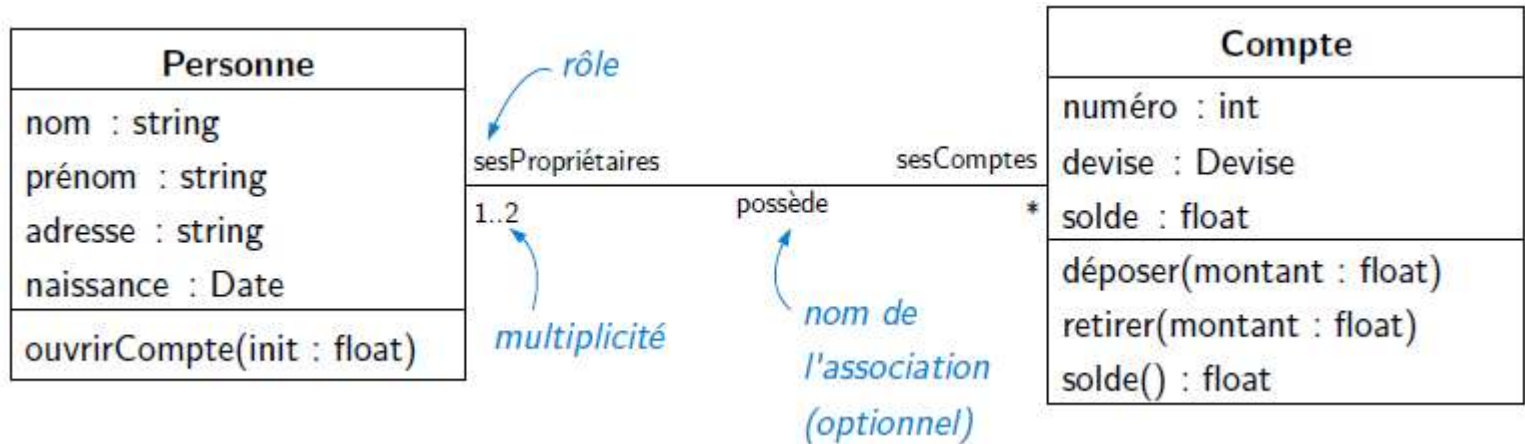
Exemple 2 : lien entre objets



Exemple 2 : lien entre objets



Exemple 2 : Association



Multiplicité

- Nombre d'objets de la classe B associés à un objet de la classe A

Exactement n



Exactement n ou m ou p



Entre n et m



Au moins n

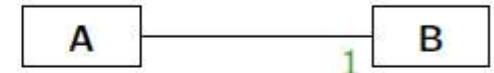


Plusieurs (0 ou plus)

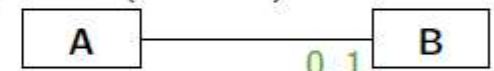


En pratique

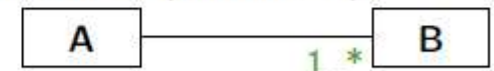
Exactement 1



Au plus 1 (0 ou 1)



Au moins 1 (jamais 0)



0 ou plus

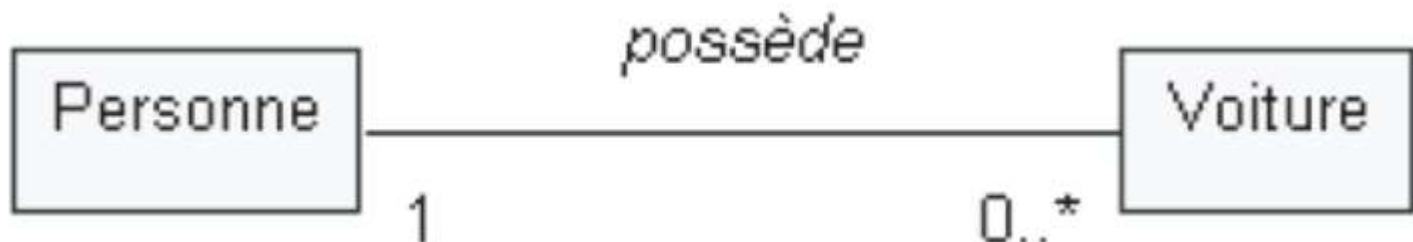


Exemple

- une personne peut posséder plusieurs voitures (entre zéro et un nombre quelconque) ; une voiture est possédée par une seule personne.

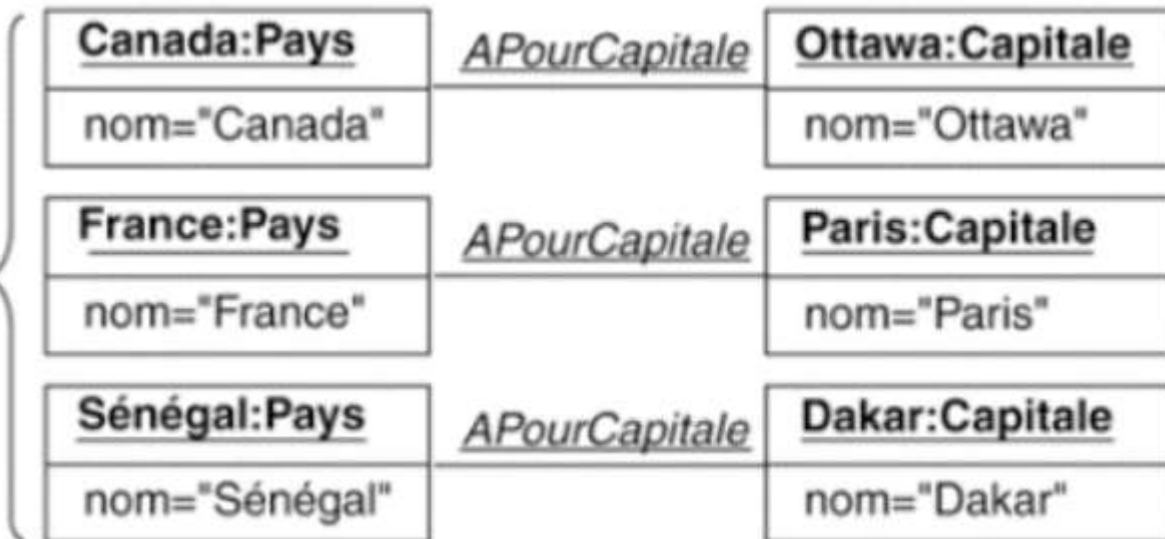
Exemple

- une personne peut posséder plusieurs voitures (entre zéro et un nombre quelconque) ; une voiture est possédée par une seule personne.

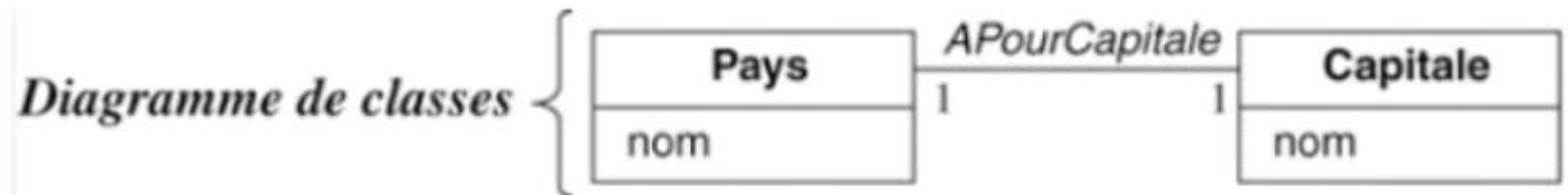


Exemple I:

Diagramme d'objets

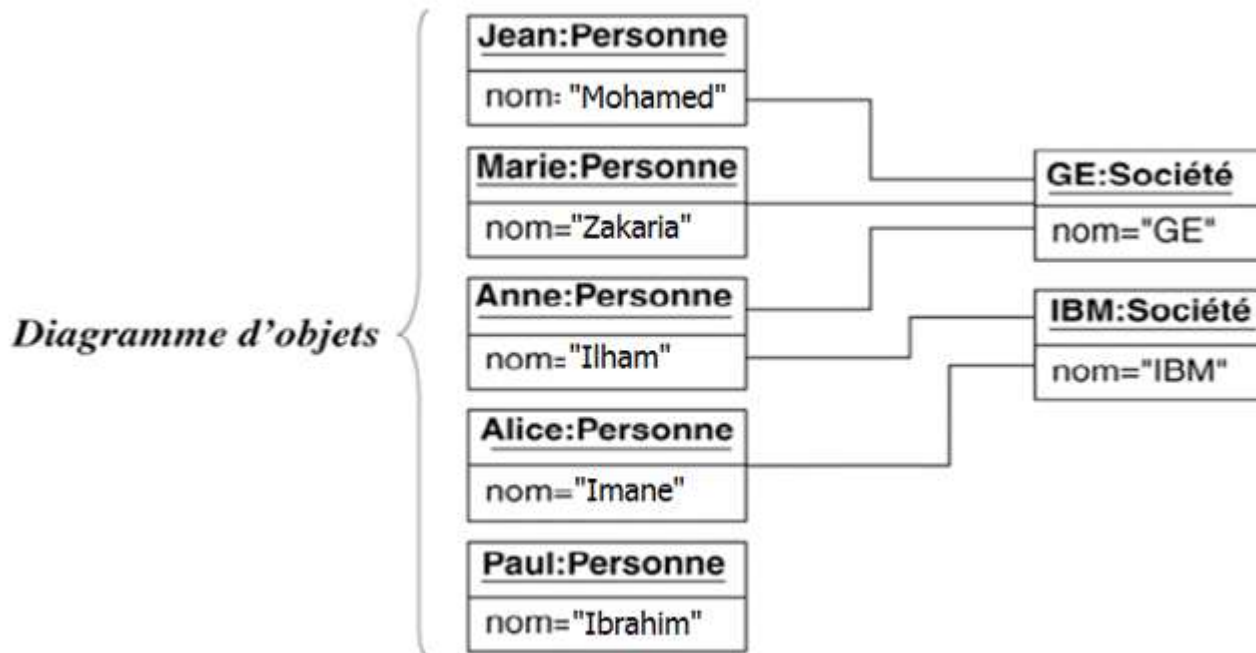


Exemple I :

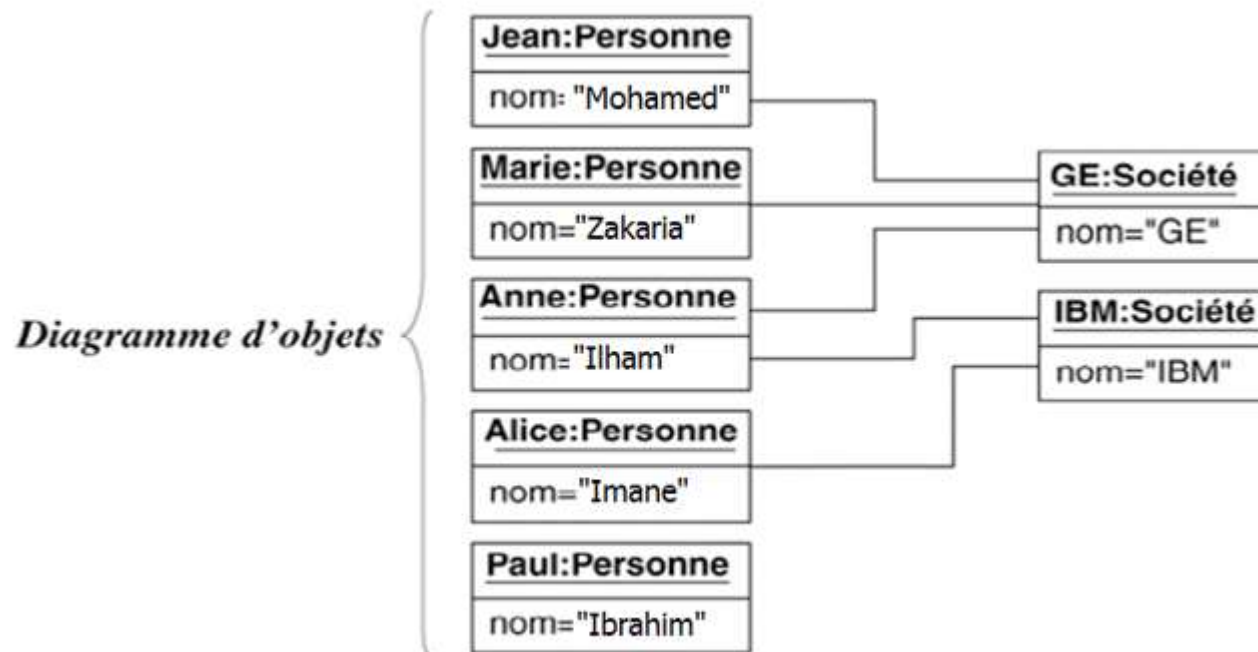
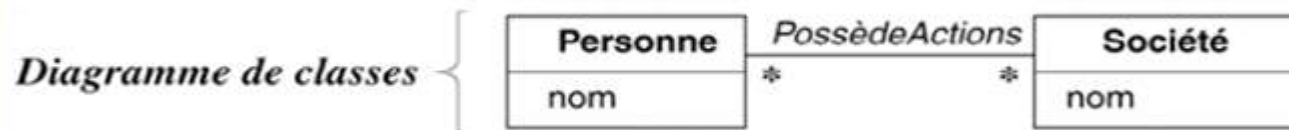


Exemple 2 :

- On suppose l'existence de plusieurs personnes qui travaillent dans une entreprise, et les personnes peuvent travailler dans plusieurs entreprises



Exemple 2 :



navigabilité

- La navigabilité indique s'il est possible de traverser une association.
- On représente graphiquement la navigabilité par une flèche du côté de la terminaison navigable et on empêche la navigabilité par une croix du côté de la terminaison non navigable.
- Par défaut, une association est navigable dans les deux sens



Exemple

à une personne sont associées ses copies d'examen mais l'inverse n'est pas possible (retrouver directement l'auteur de la copie d'examen n'est pas possible notamment avant la correction de la copie).

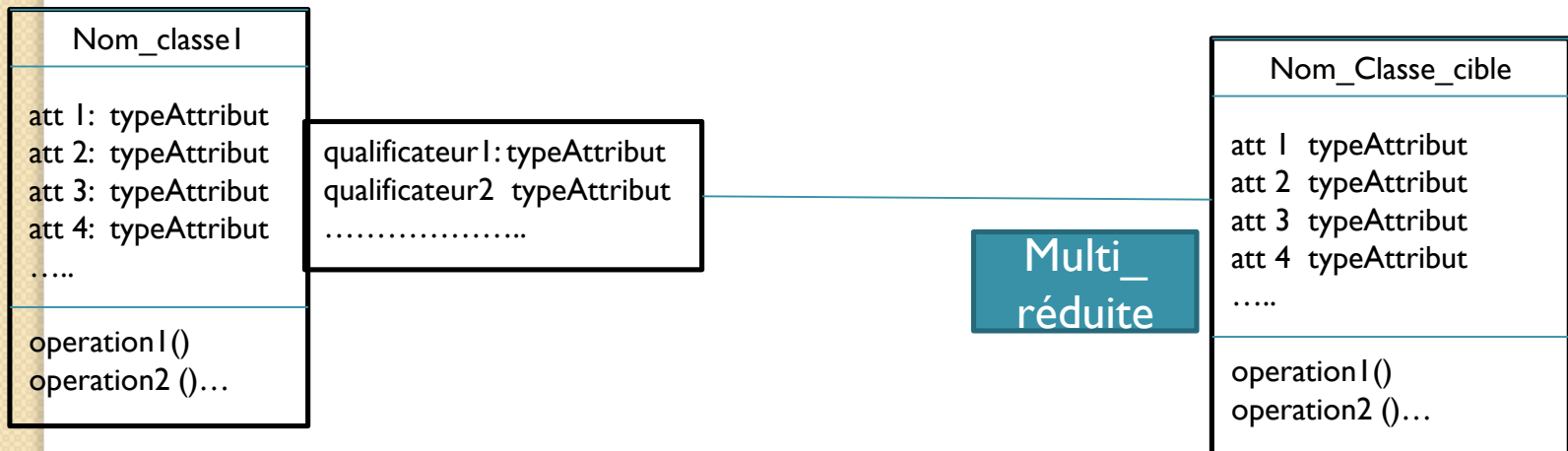
Exemple

à une personne sont associées ses copies d'examen (entre 1 et 5 copies) mais l'inverse n'est pas possible (retrouver directement l'auteur de la copie d'examen n'est pas possible notamment avant la correction de la copie).

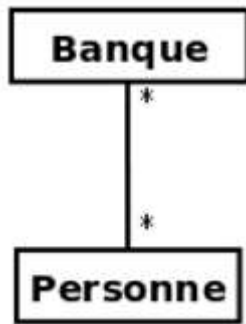


Qualification

- il est parfois préférable de restreindre la portée de l'association à **quelques éléments ciblés** (comme un ou plusieurs attributs) de la classe.
 - Ces éléments ciblés sont appelés un qualificatif.
 - Un qualificatif permet donc de sélectionner un ou des objets dans le jeu des objets d'un objet (appelé objet qualifié) relié par une association à un autre objet.
 - L'objet sélectionné par la valeur du qualificatif est appelé objet cible.
 - L'association est appelée association qualifiée.
 - Un qualificatif agit toujours sur une association dont la multiplicité est plusieurs (avant que l'association ne soit qualifiée) du côté cible.
-
- Un objet qualifié et une valeur de qualificatif génèrent un objet cible lié unique.
 - En considérant un objet qualifié, chaque valeur de qualificatif désigne un objet cible unique.

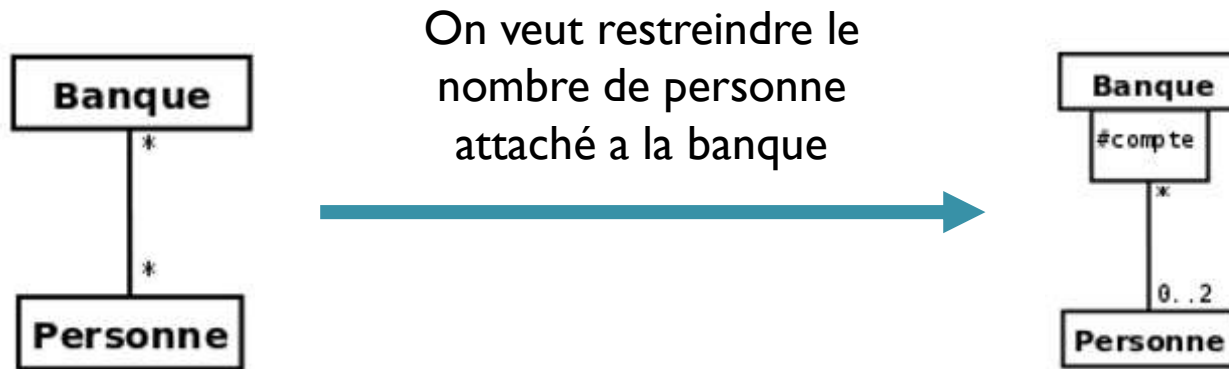


Exemple

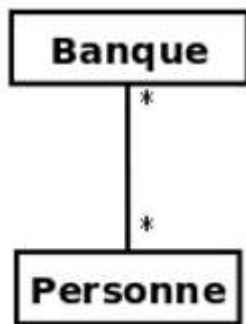


On veut restreindre le
nombre de personne
attaché a la banque

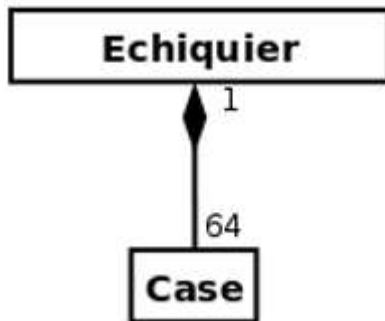
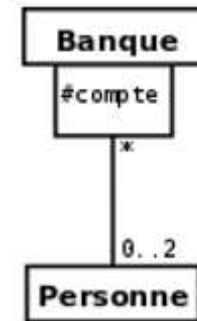
Exemple



Exemple

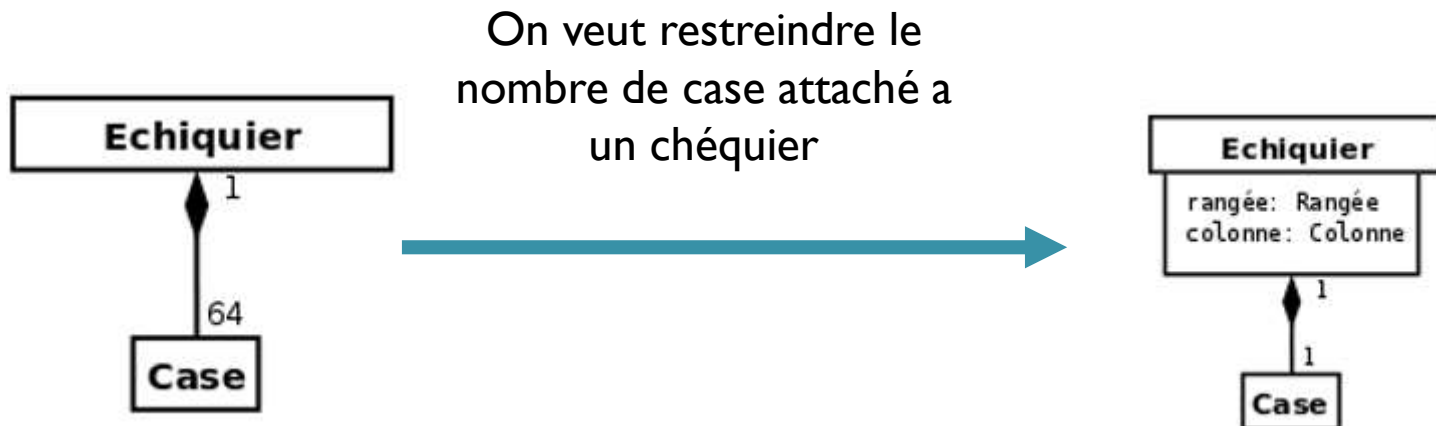
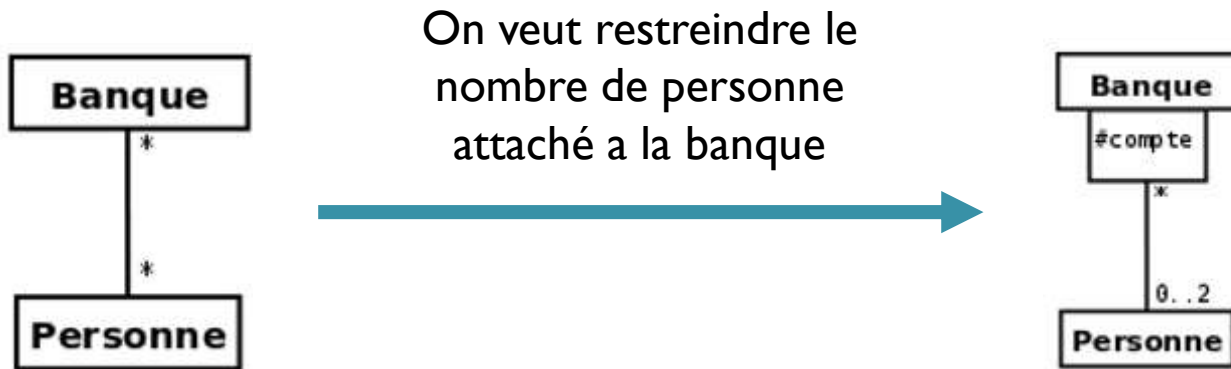


On veut restreindre le
nombre de personne
attaché a la banque



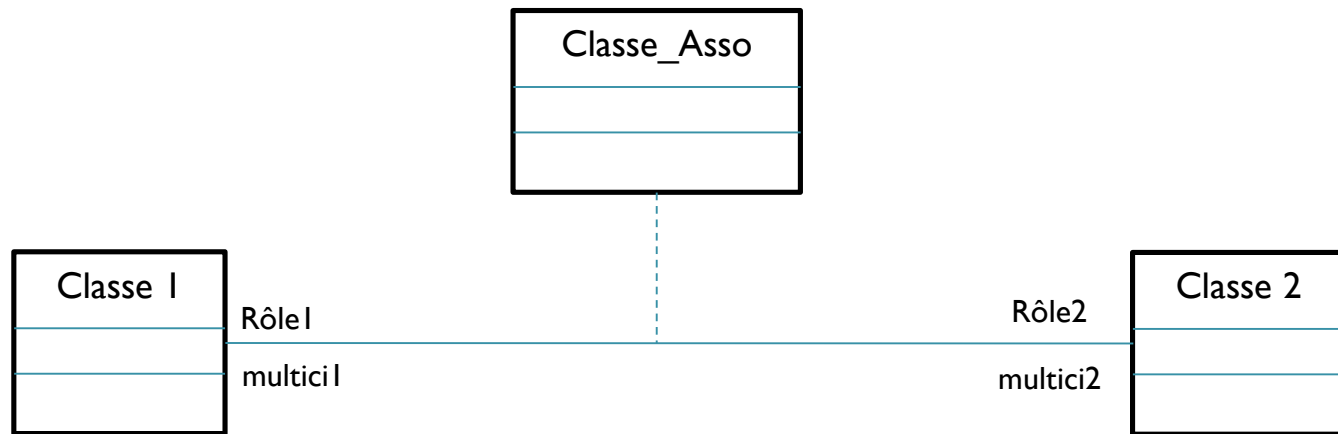
On veut restreindre le
nombre de case attaché a
un échiquier

Exemple



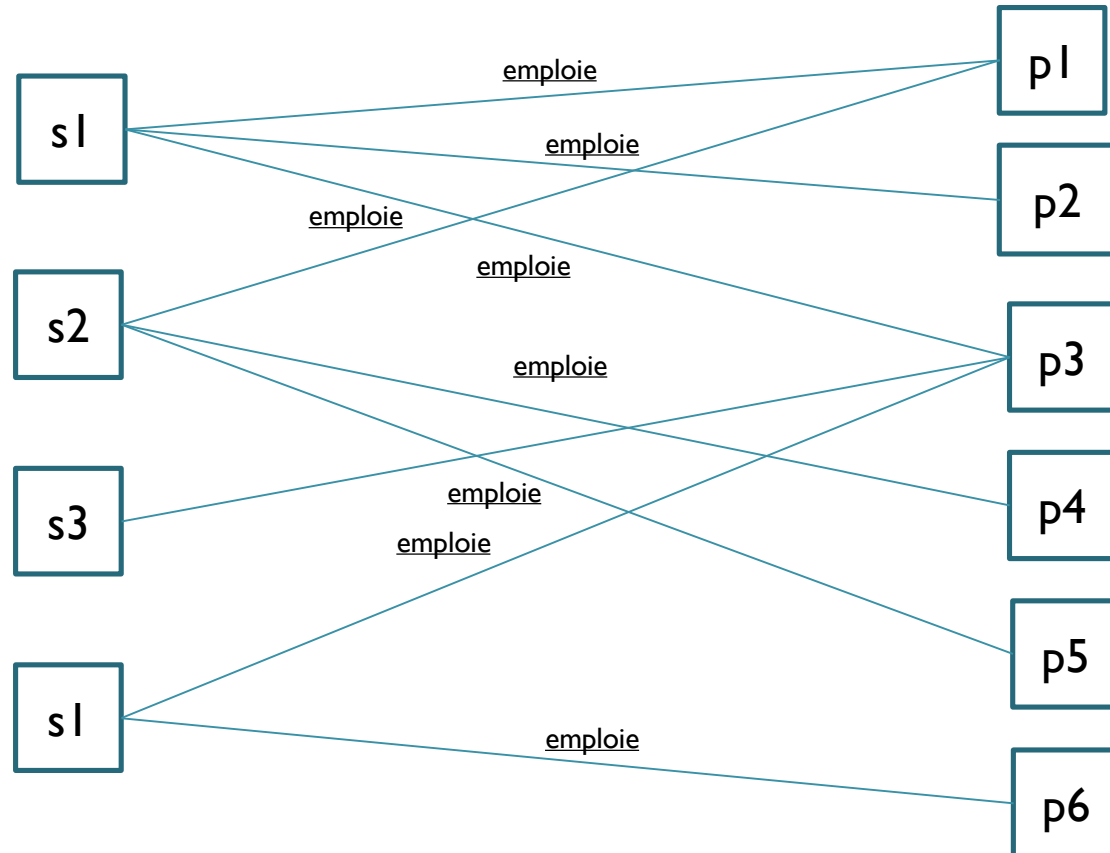
Classe-association

- Parfois, une association doit posséder des propriétés.
- Une classe-association possède les caractéristiques des associations et des classes : elle se connecte à deux ou plusieurs classes et possède également des attributs et des opérations.
- Une classe-association est caractérisée par un trait discontinu entre la classe et l'association qu'elle représente

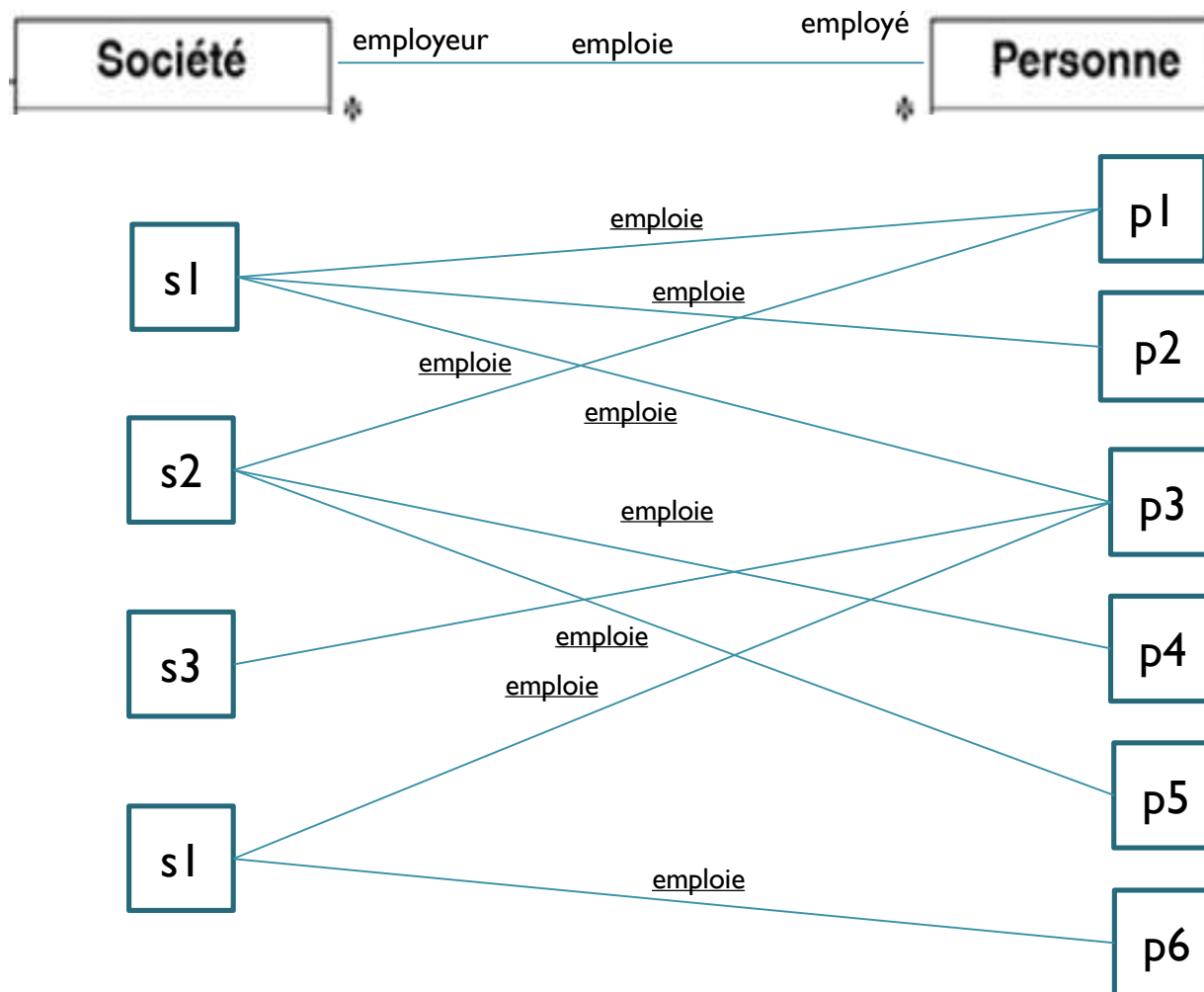


Exemple I

- Société emploie plusieurs personne, une personne peut travailler dans plusieurs sociétés

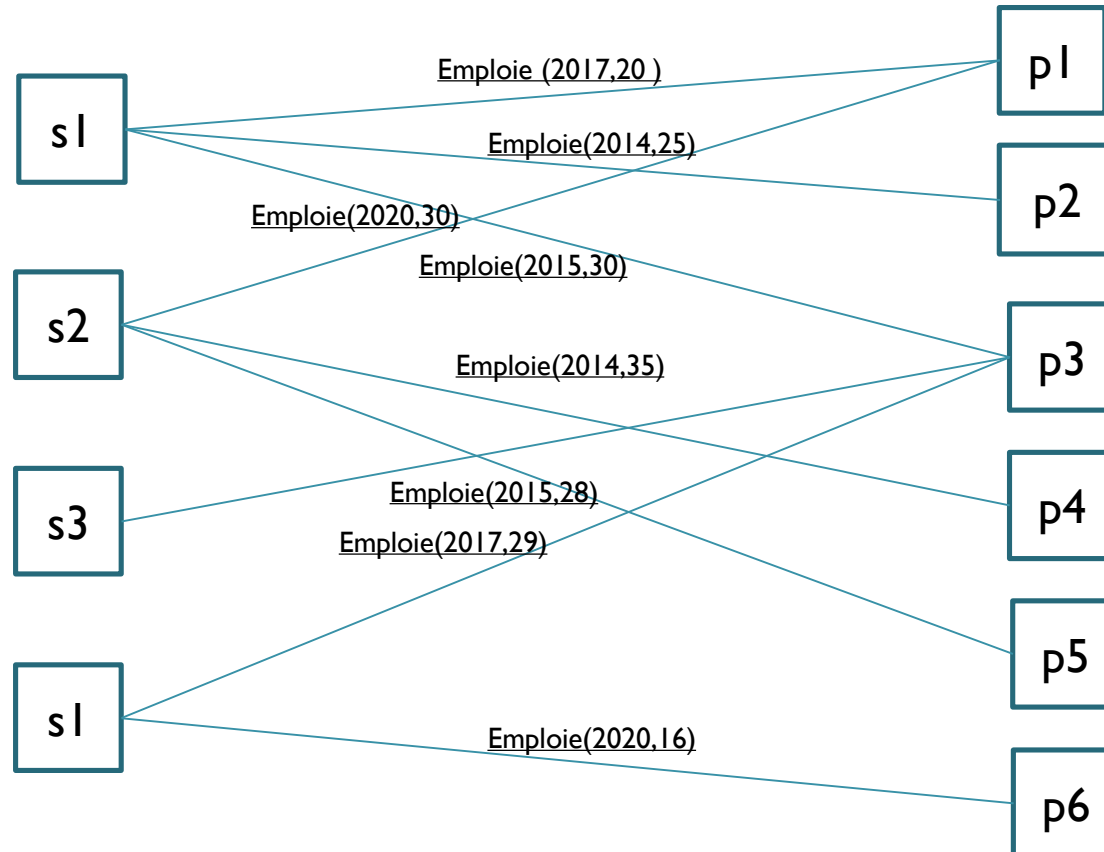


Exemple I

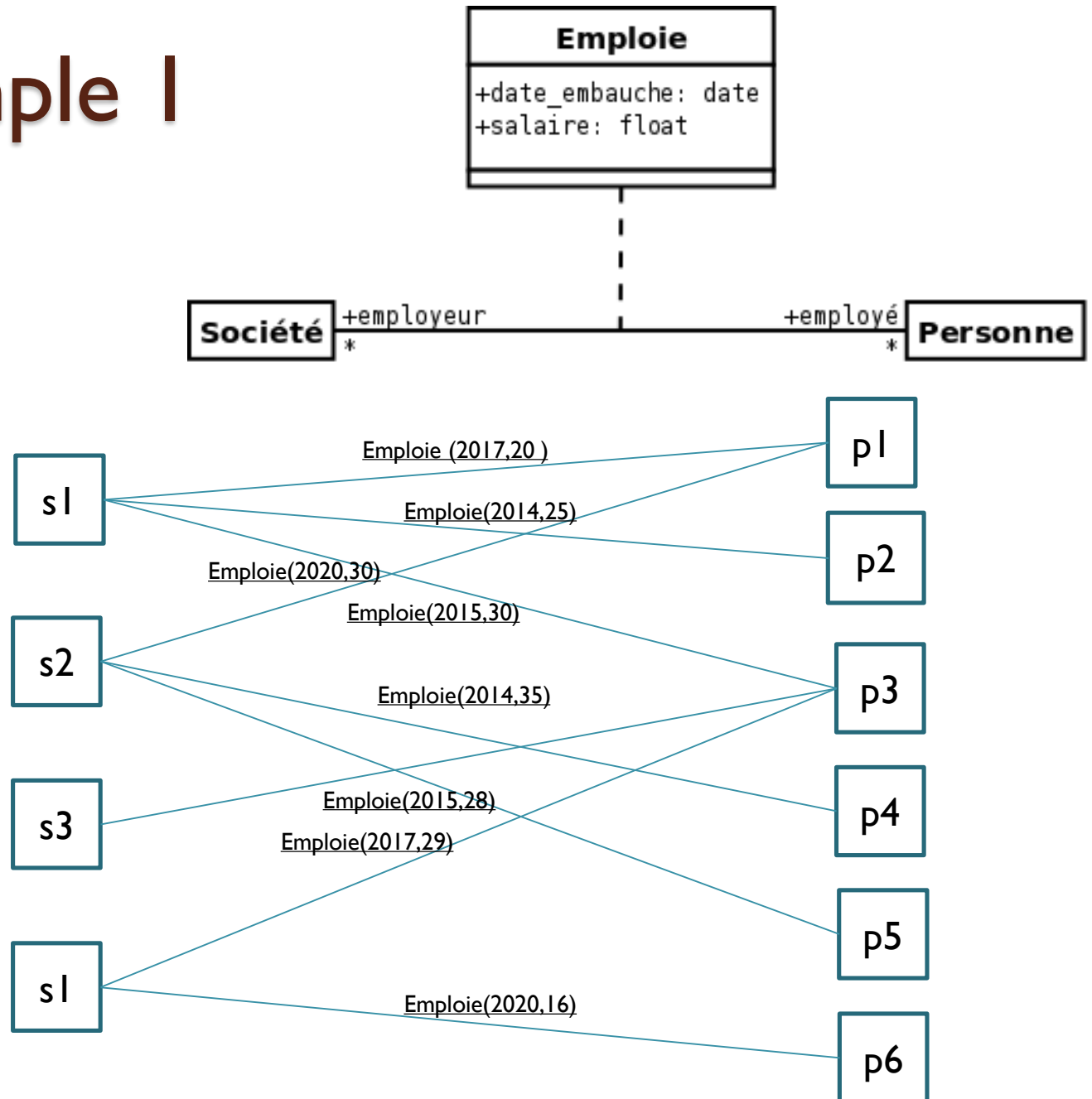


Exemple I

- On veut ajouter la date et le salaire de l'employé dans la société

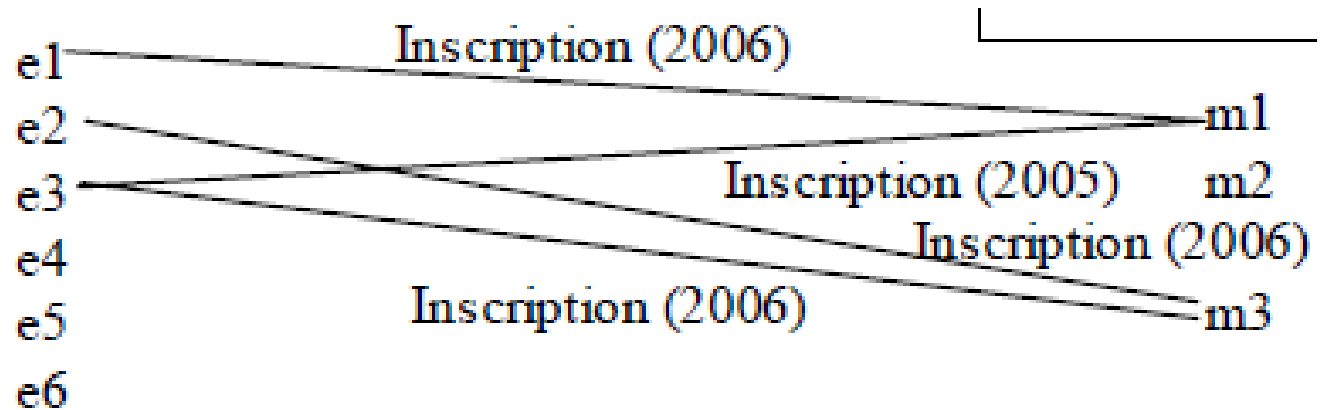


Exemple 1



Exemple 2

- Un étudiant peut s'inscrire dans plusieurs matières, dans une matière on peut inscrire plusieurs étudiants



Exemple 2

